

- ✓ Production Ready
- ✓ Secure
- ✓ Scalable
- ✓ Reliable
- ✓ Observable



MASTER
KUBERNETES
IN PRODUCTION

KUBERNETES

ULTIMATE CHEAT SHEET

THE COMPLETE QUICK REFERENCE FOR DEVOPS, ENGINEERS
AND PLATFORM PROFESSIONALS



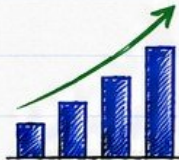
DEPLOY

Deploy and run containerized applications with confidence.



SECURE

Secure workloads, secrets, access and network at every layer.



SCALE

Scale applications and infrastructure to meet any demand.



STORAGE

Manage storage like a pro with volumes and stateful workloads.



OBSERVE

Monitor, log and troubleshoot with clarity and precision.

“EVERYTHING YOU NEED TO RUN, SECURE, AND SCALE KUBERNETES LIKE A PRO.”

”



CONNECT WITH VERIQTA

youtube.com/@veriqta

github.com/veriqta

linkedin.com/in/veriqta

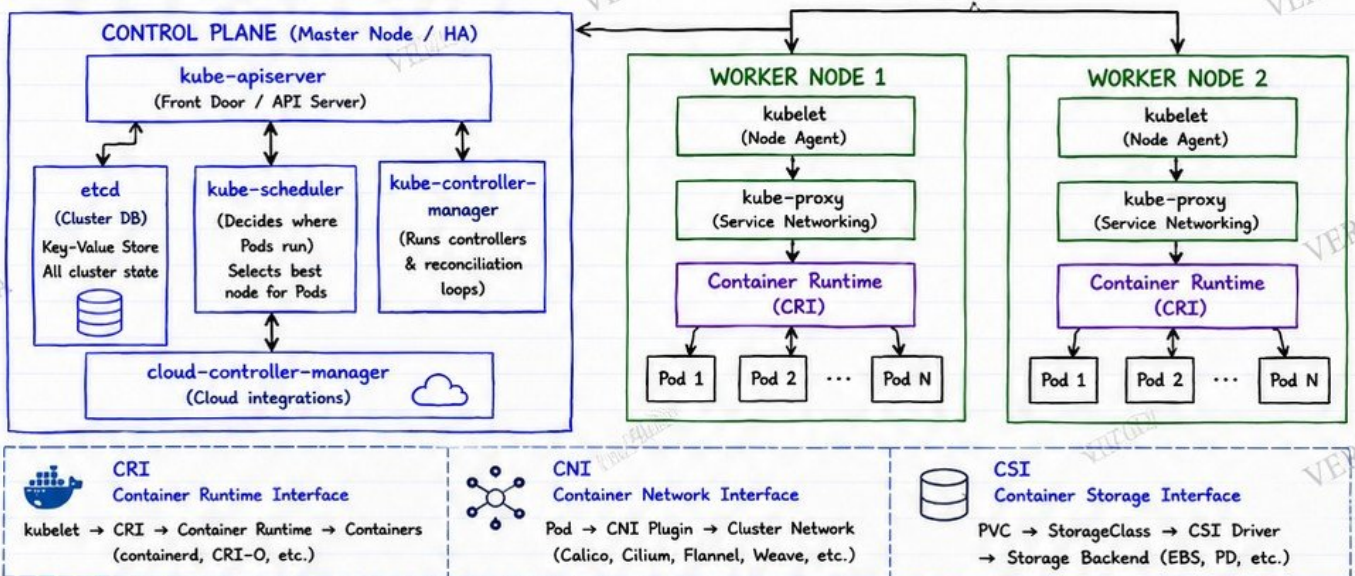
instagram.com/veriqta

twitter.com/veriqta

t.me/veriqta

KUBERNETES ARCHITECTURE AND CONTROL PLANE

1. KUBERNETES CLUSTER ARCHITECTURE



2. CONTROL PLANE COMPONENTS

- kube-apiserver:** Front door to the cluster. Handles all API requests, authN, authZ, admission, validation and persistence to etcd.
- etcd:** Highly consistent key-value store that holds all cluster state and configuration.
- kube-scheduler:** Watches for unscheduled Pods and selects the best node based on policies.
- kube-controller-manager:** Runs controllers that continuously reconcile actual state to desired state.
- cloud-controller-manager:** Integrates with cloud provider APIs for nodes, routes, load balancers, etc.

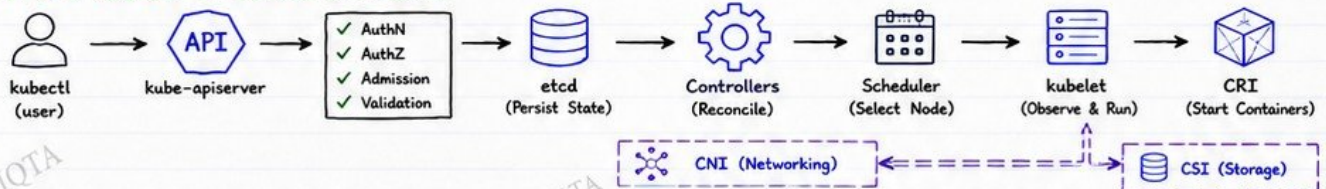
What is Kubernetes?

Kubernetes is an open-source platform that automates deployment, scaling, and operations of containerized applications.

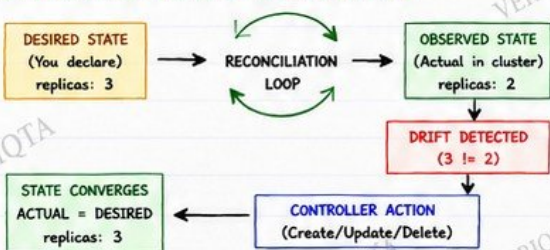
3. WORKER NODE COMPONENTS

- kubelet:** Ensures Pods assigned to the node are running and healthy. Talks to container runtime via CRI.
- kube-proxy:** Implements Service networking on the node (iptables / IPVS / eBPF depending on configuration).
- Container Runtime:** Pulls images, runs containers, manages container lifecycle.
- Pods:** Smallest deployable units running one or more containers.

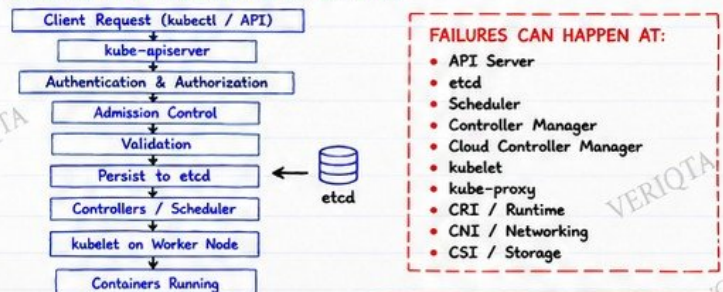
4. API REQUEST LIFECYCLE (HIGH LEVEL)



5. DESIRED STATE AND RECONCILIATION



6. CONTROL PLANE REQUEST-FLOW DIAGRAM



7. ESSENTIAL ARCHITECTURE COMMANDS

kubectl cluster-info	# Basic cluster info	kubectl describe node <node-name>	# Node details
kubectl get nodes -o wide	# List nodes	kubectl top nodes	# Node metrics
kubectl get pods -A -o wide	# List all pods	kubectl top pods -A	# Pod metrics
kubectl get componentstatuses	# Control plane components health	kubectl get csdrivers	# CSI drivers
kubectl get --raw=/readyz?verbose	# API server readiness details	kubectl get storageclasses	# Storage classes
kubectl get events -A --sort-by=.metadata.creationTimestamp	# Recent events (newest first)		
kubectl api-resources	# All API resources		

★ PRODUCTION NOTE

Understand which component owns each failure before troubleshooting the workload. Example: A Pending Pod might be a Scheduler issue, a Storage issue, a Node resource issue, or a CNI issue — not always the application.



kubectl Ultimate Command Reference

1. CLUSTER AND CONTEXT COMMANDS

<code>kubectl config view</code>	Show current kubeconfig
<code>kubectl config get-contexts</code>	List all contexts
<code>kubectl config current-context</code>	Show current context
<code>kubectl config use-context <name></code>	Switch to a context
<code>kubectl config set-context <name> --namespace=<ns></code>	Set namespace for a context
<code>kubectl cluster-info</code>	Display cluster info
<code>kubectl cluster-info dump</code>	Dump cluster info (debugging)

2. RESOURCE GET COMMANDS

<code>kubectl get all</code>	Get all resources in namespace
<code>kubectl get pods</code>	Get pods
<code>kubectl get pod <name></code>	Get specific pod
<code>kubectl get pods -o wide</code>	Get pods with more details
<code>kubectl get nodes</code>	Get nodes
<code>kubectl get svc</code>	Get services
<code>kubectl get deployments</code>	Get deployments
<code>kubectl get <resource></code>	Get any resource
<code>kubectl get all -A</code>	Get all resources in all namespaces
<code>kubectl get pods --show-labels</code>	Show labels for pods

3. DESCRIBE AND EXPLAIN

<code>kubectl describe <resource> <name></code>	Show detailed info
<code>kubectl describe pods <name></code>	Describe a pod
<code>kubectl describe node <name></code>	Describe a node
<code>kubectl describe svc <name></code>	Describe a service
<code>kubectl describe events</code>	Show events
<code>kubectl explain <resource></code>	Get documentation for a resource
<code>kubectl explain pods.spec</code>	Explain a specific field

4. CREATE COMMANDS

<code>kubectl create -f <file.yaml></code>	Create from file
<code>kubectl create deployment <name> --image=<image></code>	Create deployment
<code>kubectl create svc clusterip <name> --tcp=80:80</code>	Create ClusterIP service
<code>kubectl create namespace <name></code>	Create namespace
<code>kubectl create configmap <name> --from-literal=key=value</code>	Create configmap
<code>kubectl create secret generic <name> --from-literal=key=value</code>	Create secret

5. APPLY COMMANDS

<code>kubectl apply -f <file.yaml></code>	Apply configuration
<code>kubectl apply -k <dir></code>	Apply kustomize directory
<code>kubectl apply -f <url></code>	Apply from URL
<code>kubectl apply --server-side -f <file.yaml></code>	Server-side apply
<code>kubectl apply -f <file.yaml> --validate=false</code>	Apply without schema validation

6. DELETE COMMANDS

<code>kubectl delete -f <file.yaml></code>	Delete from file
<code>kubectl delete pod <name></code>	Delete pod
<code>kubectl delete svc <name></code>	Delete service
<code>kubectl delete deployment <name></code>	Delete deployment
<code>kubectl delete namespace <name></code>	Delete namespace
<code>kubectl delete all --all</code>	Delete all in namespace
<code>kubectl delete all --all -A</code>	Delete all in all namespaces

7. EDIT, PATCH, REPLACE

<code>kubectl edit <resource> <name></code>	Edit resource in default editor
<code>kubectl patch <resource> <name> -p '{"key":"value"}'</code>	Patch resource (strategic merge)
<code>kubectl patch <resource> <name> --type=merge -p '{"key":"value"}'</code>	Merge patch
<code>kubectl patch <resource> <name> --type=json -p '[{"op":"replace", "path":"/spec/replicas","value":3}]'</code>	JSON patch
<code>kubectl replace -f <file.yaml></code>	Replace resource
<code>kubectl replace --force -f <file.yaml></code>	Force replace (delete & recreate)

8. DIFF COMMAND

<code>kubectl diff -f <file.yaml></code>	Show diff of changes
<code>kubectl diff -k <dir></code>	Show diff of directory
<code>kubectl diff -f <file.yaml> --server-side</code>	Server-side diff



SENIOR ENGINEER TIP

Use `kubectl diff` before applying important configuration changes.

9. LOGS AND EXECUTION

<code>kubectl logs <pod></code>	Get logs from pod
<code>kubectl logs <pod> -c <container></code>	Logs from specific container
<code>kubectl logs <pod> -f</code>	Follow logs
<code>kubectl logs <pod> --previous</code>	Previous container logs
<code>kubectl exec -it <pod> -- /bin/bash</code>	Execute command in pod
<code>kubectl exec -it <pod> -c <container> -- /bin/sh</code>	Exec in specific container
<code>kubectl attach <pod> -c <container></code>	Attach to running container
<code>kubectl run <name> -it --rm --image=<image> -- /bin/sh</code>	Run temporary pod and open shell

10. FILE COPY AND PORT FORWARD

<code>kubectl cp <pod>:/path/file ./</code>	Copy file from pod
<code>kubectl cp ./file <pod>:/path/</code>	Copy file to pod
<code>kubectl port-forward <pod> 8080:80</code>	Forward local port to pod
<code>kubectl port-forward svc/<svc> 8080:80</code>	Forward local port to service
<code>kubectl port-forward deploy/<deploy> 8080:80</code>	Forward local port to deployment

11. RESOURCE USAGE (TOP)

<code>kubectl top nodes</code>	Show node CPU/Memory usage
<code>kubectl top pods</code>	Show pod CPU/Memory usage
<code>kubectl top pod <pod> --containers</code>	Show usage per container

12. AUTHORIZATION

<code>kubectl auth can-i get pods</code>	Can I get pods?
<code>kubectl auth can-i create deployments -n prod</code>	Check permission in namespace
<code>kubectl auth can-i --list</code>	List all allowed actions

13. API RESOURCES

<code>kubectl api-resources</code>	List all API resources
<code>kubectl api-resources --verbs=list --namespaced=true</code>	List namespaced resources
<code>kubectl api-resources -o wide</code>	List with more details

14. JSONPATH QUERIES

<code>kubectl get pods -o jsonpath='{.items[*].metadata.name}'</code>	Get pod names
<code>kubectl get pods -o jsonpath='{.items[*].status.phase}'</code>	Get pod phases
<code>kubectl get pod <name> -o jsonpath='{.status.podIP}'</code>	Get pod IP
<code>kubectl get node -o jsonpath='{range .items[*]}{.metadata.name} {status.addresses[?(@.type=="Internal")].address}{","}</code>	Get node internal IPs

15. CUSTOM COLUMNS

<code>kubectl get pods -o custom-columns=NAME:.metadata.name,IP:.status.podIP,PHASE:.status.phase</code>	
<code>kubectl get nodes -o custom-columns=NAME:.metadata.name,ROLE:.metadata.labels['node-role.kubernetes.io/role'],STATUS:.status.conditions[?(@.type=="Ready")].status</code>	
<code>kubectl get pods --no-headers -o custom-columns=":metadata.name"</code>	Only names, no headers

16. LABEL AND FIELD SELECTORS

Label Selectors	
<code>kubectl get pods -l app=web</code>	Pods with label app=web
<code>kubectl get pods -l app=web,env=prod</code>	Multiple labels (AND)
<code>kubectl get pods -l 'app in (web,api)'</code>	Label in list
<code>kubectl get pods -l 'env notin (dev,test)'</code>	Label not in list
Field Selectors	
<code>kubectl get pods --field-selector=status.phase=Running</code>	Pods in Running phase
<code>kubectl get pods --field-selector=spec.nodeName=node1</code>	Pods on a specific node

17. OUTPUT FORMATS

<code>kubectl get pods -o wide</code>	Wide output
<code>kubectl get pods -o yaml</code>	YAML output
<code>kubectl get pods -o json</code>	JSON output
<code>kubectl get pods -o name</code>	Resource names only
<code>kubectl get pods -o json jq</code>	JSON with jq
<code>kubectl get pods -o yaml > out.yaml</code>	Save YAML to file

18. NAMESPACE SHORTCUTS

<code>kubectl get pods -n <ns></code>	Use specific namespace
<code>kubectl config set-context --current --namespace=<ns></code>	Set default namespace
<code>kubectl get pods --all-namespaces (-A)</code>	All namespaces
<code>kubectl get pods -n default</code>	Explicit default namespace

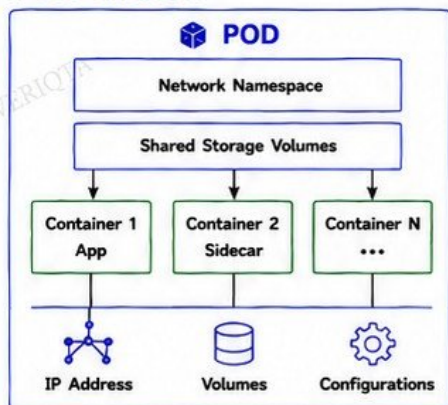
19. DEBUGGING SHORTCUTS

<code>kubectl get pods --all-namespaces</code>	<code>kubectl exec -it <pod> -- sh</code>	<code>kubectl get pods -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName</code>
<code>kubectl get events --sort-by=.lastTimestamp</code>	<code>kubectl top nodes</code>	<code>kubectl get pods --field-selector=status.phase=Running</code>
<code>kubectl describe pod <pod></code>	<code>kubectl top pods</code>	
<code>kubectl logs <pod> -f</code>	<code>kubectl get pods -o wide</code>	



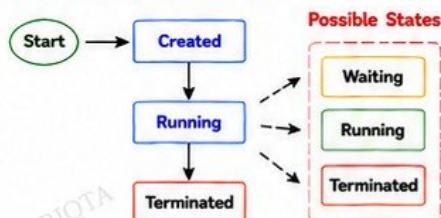
Pods and Container Lifecycle

1. POD ANATOMY



- Pod is the smallest deployable unit in Kubernetes.
- Containers in a Pod share network and storage.
- Pods are ephemeral – replaced, not rescheduled.

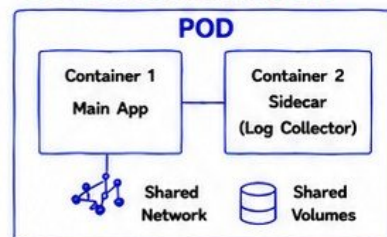
6. CONTAINER LIFECYCLE



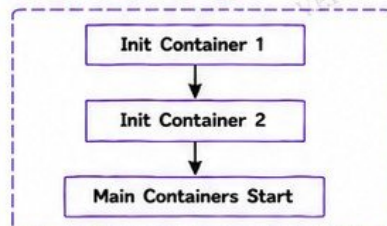
2. POD YAML STRUCTURE (EXAMPLE)

```
apiVersion: v1
kind: Pod
metadata:
  name: sample-pod
  labels:
    app: myapp
spec:
  containers:
    - name: app
      image: nginx:1.25
      ports:
        - containerPort: 80
      env:
        - name: ENV
          value: production
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
      livenessProbe:
        httpGet:
          path: /healthz
          port: 80
        initialDelaySeconds: 10
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /ready
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
      lifecycle:
        preStop:
          exec:
            command: ["/bin/sh", "-c", "sleep 10"]
      restartPolicy: Always
      terminationGracePeriodSeconds: 30
```

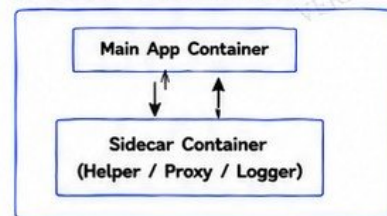
3. MULTI-CONTAINER PODS



4. INIT CONTAINERS



5. SIDECAR CONTAINERS



7. POD PHASES

Phase	Description
Pending	Pod accepted by the cluster, waiting for scheduling
Running	Pod bound to a node, containers are running
Succeeded	All containers in the Pod terminated successfully
Failed	All containers terminated and at least one failed
Unknown	Pod status cannot be obtained

8. CONTAINER STATES

State	Description
Waiting	Container is not yet running (image pull, etc.)
Running	Container is running without issues
Terminated	Container has stopped

9. RESTART POLICIES

Policy	Description
Always	Always restart the container
OnFailure	Restart only if the container exits with error
Never	Never restart the container

10. COMMANDS & ARGUMENTS

```
containers:
  - name: app
    image: myapp:latest
    command: ["/usr/bin/app"]
    args: ["--port", "8880", "--env", "prod"]
```

11. ENVIRONMENT VARIABLES

```
env:
  - name: LOG_LEVEL
    value: "info"
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: db-secret
        key: url
```

12. RESOURCE REQUESTS & LIMITS

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "1Gi"
```

15. terminationGracePeriodSeconds

Time in seconds given to the Pod to terminate gracefully.

```
spec:
  terminationGracePeriodSeconds: 30
```

18. STATIC PODS

- Managed directly by kubelet on a node.
- Defined by manifest files on the node.
- Cannot be created via API server.
- Used for control plane components.

`/etc/kubernetes/manifests/`

13. PROBES

Liveness Probe Checks if container is alive. If it fails, container is restarted. Common types: - httpGet - exec - tcpSocket	Readiness Probe Checks if container is ready to receive traffic. If it fails, Pod is removed from Service endpoints. Common types: - httpGet - exec - tcpSocket	Startup Probe Checks if application has started successfully. Disables liveness/readiness until startup succeeds. Common types: - httpGet - exec - tcpSocket
---	--	---

14. LIFECYCLE HOOKS

```
lifecycle:
  postStart:
    exec:
      command: ["/bin/sh", "-c", "echo started"]
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 10"]
```

16. GRACEFUL TERMINATION FLOW



19. EPHEMERAL CONTAINERS

- Added to a running Pod for debugging.
- Do not restart when they exit.
- Ideal for troubleshooting production issues.

```
kubectl debug -it <pod-name> --image=busybox
--target=<container-name> -- /bin/sh
```

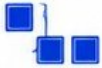
20. DEBUGGING SHORTCUTS

```
kubectl get pods -A -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name> -c <container>
kubectl exec -it <pod-name> -c <container> -- /bin/sh
kubectl top pod <pod-name>
kubectl get events --sort-by=.lastTimestamp
```

DEBUGGING INSIGHT: A Running Pod does not automatically mean the application is healthy or ready for traffic.

WORKLOAD CONTROLLERS

1. REPLICASETS



- Ensures a stable set of replica Pods are running at any time.
- No update strategy.
- Used directly by Deployments.

2. DEPLOYMENTS



- Manages ReplicaSets and provides declarative updates.
- Supports rolling updates and rollbacks.
- Recommended for stateless applications.

3. STATEFULSETS



- Manages stateful applications.
- Provides stable network identity and persistent storage.
- Ordered and graceful deployment.

4. DAEMONSETS



- Ensures a Pod runs on all (or selected) nodes.
- Used for node-level agents and monitoring tools.
- Automatically adapts to node changes.

5. JOBS



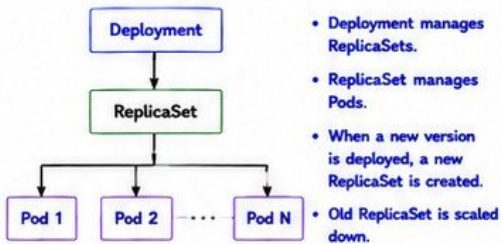
- Runs Pods to completion for batch tasks.
- Supports retries on failure.
- Does not run on a schedule.

6. CRONJOBS

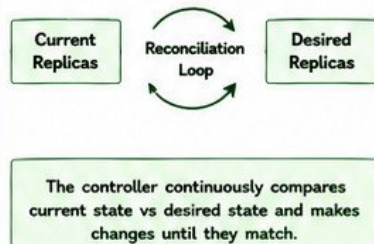


- Runs Jobs on a schedule.
- Supports concurrency policies.
- Ideal for periodic batch tasks.

7. CONTROLLER RELATIONSHIPS



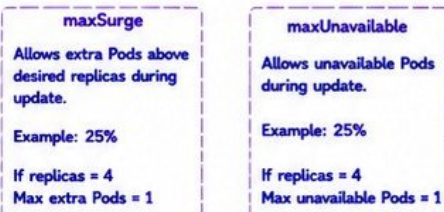
8. DESIRED REPLICAS



9. ROLLING UPDATES & ROLLSBACKS



10. DEPLOYMENT STRATEGY SETTINGS



11. DEPLOYMENT HISTORY

- `kubectl rollout history deployment <name>`
View revision history
- `kubectl rollout undo deployment <name>`
Rollback to previous revision
- `kubectl rollout status deployment <name>`
Monitor rollout progress
- Each change creates a new revision.

12. STATEFUL IDENTITY & ORDERED DEPLOYMENT

Stateful Identity

Pod Identity

web-0 web-1 web-2

Stable DNS

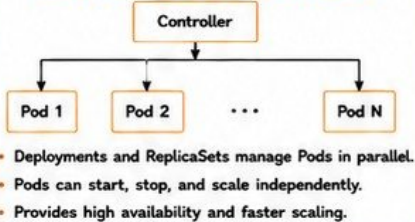
web-0.svc, web-1.svc, web-2.svc

Ordered Deployment

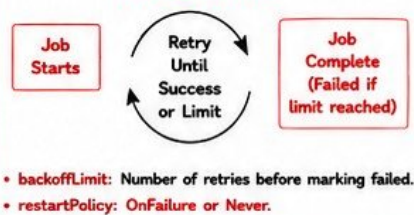
Create: web-0 → web-1 → web-2

Delete: web-2 → web-1 → web-0

13. PARALLEL POD MANAGEMENT



14. JOB RETRIES



15. CRONJOB CONCURRENCY POLICIES

Allow	Allows concurrent Jobs. New Job starts even if previous is running.
Forbid	Skips new Job if previous is still running.
Replace	Terminates previous Job and starts new one.

Set using: `concurrencyPolicy: Allow | Forbid | Replace`

16. WORKLOAD SELECTION DECISION TABLE

Workload Type	Recommended Controller	Use Case	State	Persistence	Scheduling	Update Strategy
Stateless Application	Deployment	Web apps, APIs, Microservices	Stateless	No	Any Node	Rolling Updates
Stateful Application	StatefulSet	Databases, Zookeeper, Kafka, etc.	Stateful	Yes	Ordered Pods	Ordered Rolling
Node-Level Agent	DaemonSet	Logging, Monitoring, Node agents	Stateless	Optional	All or Selected Nodes	Rolling by Node
Batch Task (One-time)	Job	Data processing, Migrations, Scripts	Stateless	No	Any Node	Run to Completion
Scheduled Task	CronJob	Backups, Reports, Cleanup tasks	Stateless	No	Any Node	Scheduled Jobs
Simple Replica Management	ReplicaSet	Manually managing Pod replicas	Stateless	No	Any Node	No Update Strategy



PRODUCTION NOTE: Select controllers based on workload behavior, not convenience.



VERIQTA

youtube.com/@veriqta

github.com/veriqta

linkedin.com/in/veriqta

instagram.com/veriqta

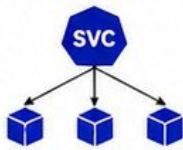
twitter.com/veriqta

t.me/veriqta

Services and Service Discovery

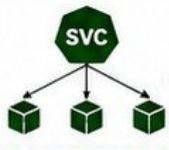
1. SERVICE TYPES

1.1 CLUSTERIP



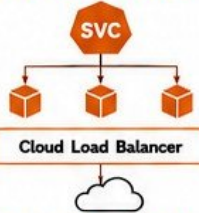
- Default service type
- Exposes service on a cluster-internal IP
- Only reachable inside the cluster
- Stable virtual IP

1.2 NODEPORT



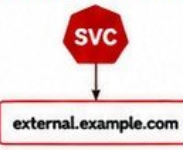
- Exposes service on each node's IP at a static port
- Reachable from outside using NodeIP:NodePort
- Traffic goes to NodePort then to service

1.3 LOADBALANCER



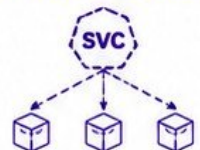
- Provisions external load balancer (cloud provider)
- Exposes service externally
- Traffic flows: LB → Node → Service → Pods

1.4 EXTERNALNAME



- Maps service to an external DNS name
- No proxy or load balancing
- Returns CNAME record

1.5 HEADLESS SERVICE



- Set clusterIP: None
- No virtual IP
- Returns Pod IPs directly
- Used for StatefulSets, custom discovery

2. SERVICE SELECTORS

Service selects Pods using labels.

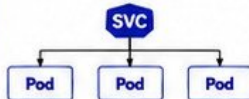
Example:

selector:
app: myapp
tier: backend



Only Pods with matching labels are part of the Service.

3. ENDPOINTS



- Endpoints is a list of Pod IPs and ports backing a Service
- Legacy API object
- Automatically created and updated by Kubernetes

4. ENDPOINTSlices



- Scalable replacement for Endpoints
- Supports large numbers of Pods
- More efficient updates
- Recommended in modern clusters

5. SERVICE DNS

Service DNS Format:

`<service-name>.<namespace>.svc.cluster.local`

- Example: myapp.default.svc.cluster.local
- Resolved by CoreDNS inside the cluster
- Works across all namespaces

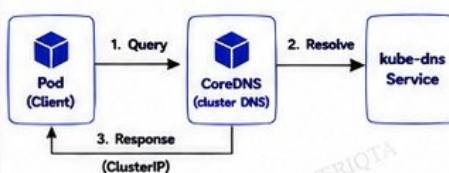
6. COREDNS



CoreDNS

- Default DNS server in Kubernetes
- Handles service discovery
- Resolves service names to ClusterIP
- Configurable via ConfigMaps

7. CLUSTER DNS RESOLUTION FLOW



8. SERVICE ENVIRONMENT VARIABLES

Variable	Description
<code><SERVICE_NAME>_SERVICE_HOST</code>	ClusterIP of the service
<code><SERVICE_NAME>_SERVICE_PORT</code>	Port of the service
<code><SERVICE_NAME>_PORT</code>	Port of the service

`<SERVICE_NAME>` is in UPPERCASE with _

9. SESSION AFFINITY

Routes requests from the same client IP to the same Pod.

Types:

- None (default)
- ClientIP

Useful for stateful applications without external session store.



10. EXTERNALTRAFFICPOLICY

Controls how external traffic is routed.

Policy	Behavior
Cluster (default)	Traffic may be routed to any node. Source IP may be lost.
Local	Traffic only to node with local endpoint. Source IP preserved.

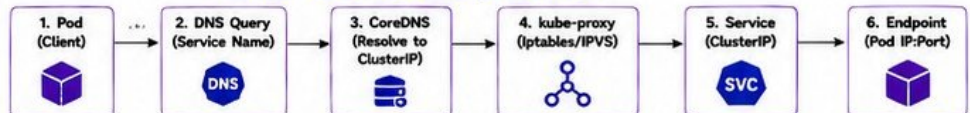
11. INTERNAL VS EXTERNAL TRAFFIC

Internal Traffic	External Traffic
• Pod → Service (ClusterIP) • Service → Service • In-cluster only • Uses ClusterIP • Fast and private	• From outside the cluster • Uses NodePort / LoadBalancer • Passes through nodes/LB • Consider security, firewalls • May need Ingress or WAF

12. SERVICE TROUBLESHOOTING CHECKLIST

- ☒ Service has correct selector
- ☒ Pods have matching labels
- ☒ Pods are in Ready state
- ☒ Endpoints / EndpointSlices exist
- ☒ Service type and ports are correct
- ☒ Network policies / firewalls allow traffic
- ☒ DNS resolution works
- ☒ Check Service events: `kubectl describe svc <name>`

13. POD-TO-SERVICE REQUEST FLOW



How it works:

1. Pod sends request using service DNS name.
2. DNS query is sent to CoreDNS.
3. CoreDNS resolves service name to ClusterIP.
4. kube-proxy forwards traffic to a healthy endpoint.
5. Traffic reaches the Service virtual IP.
6. Request is delivered to one of the backend Pods.

14. PRODUCTION FAILURE SCENARIO

Service Selector

app: myapp
tier: backend

Pods Labels

app: myapp
tier: api

Result

✗ No Endpoints

Service has no matching Pods and becomes unreachable!

Fix

✓ Match Labels

Update Pod labels or Service selector to match and restore connectivity.



PRODUCTION FAILURE:

Healthy Pods can still be unreachable when Service selectors do not match Pod labels.



VERIQTA

youtube.com/@veriqta

github.com/veriqta

linkedin.com/in/veriqta

instagram.com/veriqta

twitter.com/veriqta

t.me/veriqta

Ingress, Gateway API and External Traffic

1. INGRESS RESOURCES



- Exposes HTTP(S) routes from outside the cluster to Services.
- Defined by Ingress objects.
- Rules map hosts and paths to Services.
- Supports TLS and annotations.
- Works with an Ingress Controller.

2. INGRESS CONTROLLERS



- Watches Ingress resources.
- Configures the data plane (LB/Proxy).
- Examples: NGINX Ingress, Traefik, HAProxy, Istio.
- Must be installed in the cluster.

3. INGRESSCLASS



- Defines which Ingress Controller should handle an Ingress.
- Enables multiple controllers.
- Set via `ingressClassName` or annotation.

4. HOST-BASED & PATH-BASED ROUTING



Host-Based Routing

- Routes based on domain name.

Path-Based Routing

- Routes based on URL path.
- Allows many apps under same domain.

5. TLS TERMINATION & CERTIFICATES



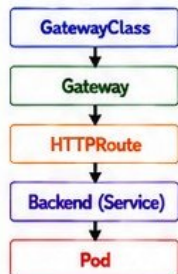
- TLS can terminate at the Ingress Controller or Load Balancer.
- Certificates stored in Kubernetes Secrets.
- Supports automatic certs via Let's Encrypt.

6. LOAD BALANCER FLOW



- Client → Internet → Load Balancer → Ingress Controller → Service → Pod
- Health checks ensure only healthy backends receive traffic.

7. GATEWAY API CONCEPTS



- Modern standard for traffic management.
- More expressive and extensible than Ingress.
- Separates traffic configuration from infrastructure.
- Supported by many controllers (NGINX, Istio, Traefik, etc.).

8. GATEWAY API OBJECTS



GatewayClass

Defines the type of infrastructure and controller that will implement Gateways.



Gateway

Represents a load balancer operated by the controller.



HTTPRoute

Defines how HTTP traffic is routed to Services.



BackendRef

References the target Service and port.

9. TRAFFIC ROUTING

- Match by host, path, headers, methods, and more.
- Support for filters (rewrite, redirect, header modify).
- Advanced policies: retry, timeout, circuit breaking.
- Splitting traffic between multiple backends.
- Canary and blue/green deployments.

10. REVERSE PROXIES



- Ingress Controllers act as reverse proxies.
- Accept external requests.
- Forward to internal Services.
- Provide TLS, routing, auth, rate limiting, WAF, caching.

11. EXTERNAL DNS CONCEPTS

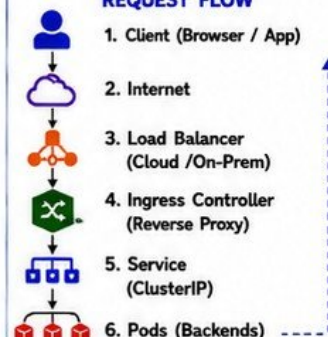


- Map domain names to the external IP/hostname of your Load Balancer.
- Common providers: Route 53, Cloud DNS, Azure DNS.
- Use ExternalDNS to automate DNS record management.
- updates.

12. COMMON HTTP ERROR TROUBLESHOOTING

ERROR	MEANING	COMMON CAUSES	TROUBLESHOOTING STEPS
404 Not Found	Resource not found	• Wrong host or path • Rule not configured	• Check Ingress/HTTPRoute rules • Verify host and path • Confirm Service exists
502 Bad Gateway	Invalid response from upstream	• Backend Pod not running • Pod crashed or endpoints	• Check Pod logs • Verify Service endpoints • Confirm readiness probes
503 Service Unavailable	Service temporarily unavailable	• No healthy endpoints • Readiness probe failing • Overloaded Pods	• Check Endpoints/EndpointSlices • Verify Pod readiness • Check resource usage
504 Gateway Timeout	Upstream did not respond in time	• Slow backend response • Network latency • Timeout too low	• Increase timeout settings • Optimize application • Check network connectivity

13. CLIENT-TO-POD REQUEST FLOW



14. SAMPLE INGRESS YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    kubernetes.io/ingress.class: "nginx"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls
  rules:
  - host: app.example.com
    http:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

15. COMMON INGRESS ANNOTATIONS

ANNOTATION	PURPOSE
<code>nginx.ingress.kubernetes.io/rewrite-target</code>	Rewrite request URI
<code>nginx.ingress.kubernetes.io/ssl-redirect</code>	Force HTTPS redirect
<code>nginx.ingress.kubernetes.io/force-ssl-redirect</code>	Redirect HTTP to HTTPS
<code>nginx.ingress.kubernetes.io/proxy-read-timeout</code>	Set backend read timeout
<code>nginx.ingress.kubernetes.io/proxy-body-size</code>	Limit request body size
<code>nginx.ingress.kubernetes.io/enable-cors</code>	Enable CORS
<code>cert-manager.io/cluster-issuer</code>	Use cert-manager issuer

16. INGRESS TROUBLESHOOTING CHECKLIST

- ☒ Ingress resource created successfully
- ☒ IngressClass exists and is valid
- ☒ Ingress Controller is running
- ☒ Load Balancer has external IP / hostname
- ☒ DNS record points to Load Balancer
- ☒ TLS certificate is valid and not expired
- ☒ Rules (host/path) are correct
- ☒ Service exists and has endpoints
- ☒ Pods are running and Ready
- ☒ Network policies / firewalls allow traffic
- ☒ Check Ingress Controller logs
- ☒ Test with curl / browser / openssl



ENTERPRISE ADVICE:

Treat the Kubernetes API objects and the actual data-plane implementation as separate troubleshooting layers.



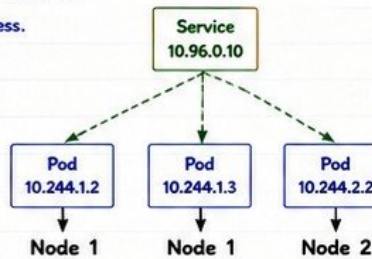
VERIQTA

[youtube.com/@veriqta](https://www.youtube.com/@veriqta)
github.com/veriqta
[linkedin.com/in/veriqta](https://www.linkedin.com/in/veriqta)
[instagram.com/veriqta](https://www.instagram.com/veriqta)
twitter.com/veriqta
t.me/veriqta

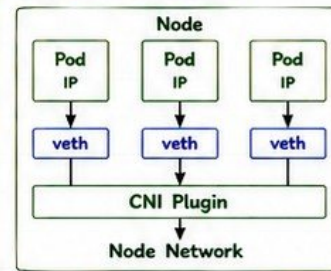
KUBERNETES NETWORKING

1. KUBERNETES NETWORK MODEL

- Every Pod gets its own IP address.
- Pods can communicate with all other Pods without NAT.
- Pods can communicate with Services using Cluster IP.
- The underlying network handles connectivity.
- The network is flat from the Pod's perspective.



2. POD NETWORKING



- Each Pod is connected to the node via a veth pair.
- The CNI plugin attaches the Pod interface to the node network.
- Each Pod gets an IP from the Pod CIDR range.
- Pods on different nodes are routable via the overlay network.

3. POD-TO-POD COMMUNICATION



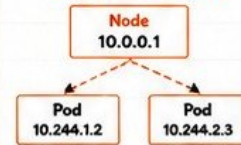
- Direct routing between Pods.
- No NAT is performed.
- Works across nodes using the overlay network.

4. POD-TO-SERVICE COMMUNICATION



- Pods access Services using Cluster IP.
- Traffic is load balanced to matching Pods.
- kube-proxy ensures this works.

5. NODE-TO-POD COMMUNICATION



- Nodes can reach Pods directly using Pod IP.
- Required for monitoring, logging, and kubelet.

6. CNI RESPONSIBILITIES



- Assign IP addresses to Pods.
- Manage Pod network interfaces.
- Enable Pod-to-Pod communication.
- Provide network policies.
- Support advanced networking features.

7. POD CIDR & SERVICE CIDR

Pod CIDR (Example): 10.244.0.0/16

Service CIDR (Example): 10.96.0.0/12

- Pod CIDR: IP range used by all Pods.
- Service CIDR: IP range used for Cluster IPs.

8. KUBE-PROXY



- kube-proxy runs on each node.
- It maintains network rules to enable Service load balancing.
- It watches Services and Endpoints.

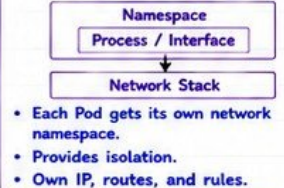
9. IPTABLES CONCEPTS

- Uses iptables rules (NAT) for Service routing.
- Traffic is DNATed to a backend Pod.
- Simple and widely compatible.
- Performance can degrade with many Services.

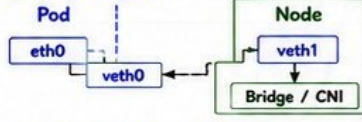
10. IPVS CONCEPTS

- Uses IPVS for load balancing.
- More scalable and high performance.
- Supports advanced load balancing algorithms.
- Recommended for large clusters.

11. NETWORK NAMESPACES

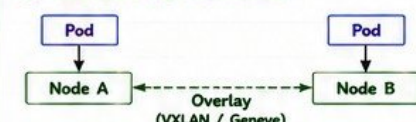


12. VETH PAIRS



- veth0 inside the Pod namespace.
- veth1 on the host in root namespace.
- Acts like a virtual ethernet cable.

13. OVERLAY NETWORKING



- Used for Pod-to-Pod communication across nodes.
- Encapsulates traffic over the underlay network.
- Common in CNI plugins like Flannel, Calico, Cilium.

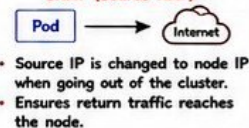
14. ROUTING



- Pods on different nodes communicate via routing.
- CNI plugin programs routes on nodes.
- Underlay network must allow this traffic.

15. SNAT AND DNAT

SNAT (Source NAT)



- Source IP is changed to node IP when going out of the cluster.
- Ensures return traffic reaches the node.

DNAT (Destination NAT)



- Service IP is translated to backend Pod IP.
- Handled by kube-proxy.

16. MTU PROBLEMS

- Overlay networks add overhead.
- If MTU is too large, packets may fragment or drop.
- Symptoms: timeouts, slow responses.
- Fix: reduce MTU on interfaces.

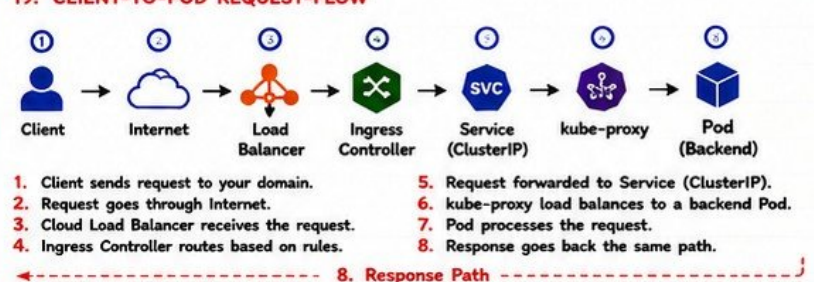
17. DNS FAILURES

- CoreDNS provides cluster DNS.
- If DNS is down, Services by name fail.
- Check: CoreDNS Pods, Service, Config.
- Use: `kubectl exec -it pod -- nslookup service-name`

18. NETWORK TROUBLESHOOTING COMMANDS

COMMAND	PURPOSE
<code>kubectl get pods -o wide</code>	Check Pod IPs and Node placement
<code>kubectl get svc -o wide</code>	Check Service IP, Ports, and Endpoints
<code>kubectl get endpoints <svc> -o wide</code>	Check backend Pod IPs
<code>kubectl exec -it <pod> -- ping <ip></code>	Test connectivity to an IP
<code>kubectl exec -it <pod> -- nslookup <name></code>	Test DNS resolution
<code>kubectl exec -it <pod> -- curl http://<ip>:<port></code>	Test HTTP connectivity
<code>ip a</code>	Show interfaces and IPs in a Pod/Node
<code>ip r</code>	Show routing table
<code>iptables -t nat -L -n -v</code>	View iptables rules (NAT)
<code>ipvsadm -ln</code>	View IPVS virtual servers
<code>tcpdump -i any</code>	Capture packets on Node/Pod
<code>ethtool <iface></code>	Check MTU and interface status

19. CLIENT-TO-POD REQUEST-FLOW



DEBUGGING INSIGHT: Trace network failures layer by layer instead of immediately blaming the CNI plugin.

— VERIQTA —

CONFIGMAPS, SECRETS AND APPLICATION CONFIGURATION



1. CONFIGMAPS

- Store non-sensitive key-value data.
- Used for application configuration.
- Can be consumed as environment variables or files.
- Not encrypted by default.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "production"
  LOG_LEVEL: "info"
```



2. SECRETS

- Store sensitive information.
- Base64 encoded by default (not encrypted).
- Used for passwords, tokens, certificates, API keys, etc.

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQ=
  API_KEY: YWJjMTIzNDU=
```



3. ENVIRONMENT VARIABLES

- Expose configuration to containers as environment variables.
- Values can come from ConfigMaps or Secrets.

```
env:
- name: APP_MODE
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: APP_MODE
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secret
      key: DB_PASSWORD
```



4. ENVFROM

- Load all keys from a ConfigMap or Secret as environment variables.
- Useful when many variables are needed.

```
envFrom:
- configMapRef:
  name: app-config
- secretRef:
  name: app-secret
```



5. VOLUME-MOUNTED CONFIGURATION

- Mount ConfigMaps or Secrets as files in a volume.
- Useful for apps expecting configuration files.
- Updates require Pod restart unless reloader is used.

```
volumes:
- name: config-volume
  configMap:
    name: app-config
- name: secret-volume
  secret:
    secretName: app-secret
volumeMounts:
- name: config-volume
  mountPath: /etc/config
- name: secret-volume
  mountPath: /etc/secret
```



6. SECRET TYPES

TYPE	DESCRIPTION
Opaque	Default key-value pairs
kubernetes.io/tls	TLS certificates and keys
kubernetes.io/dockerconfigjson	Docker registry credentials
kubernetes.io/service-account-token	Service account tokens
kubernetes.io/basic-auth	Basic auth credentials



7. SECRET CREATION

From literals:

```
kubectl create secret generic app-secret \
  --from-literal=DB_PASSWORD=pass123
```

From files:

```
kubectl create secret generic app-secret \
  --from-file=./secrets/
```

From YAML:

```
kubectl apply -f secret.yaml
```



8. UPDATING CONFIGURATION

- ConfigMaps and Secrets are immutable in Pods.
- Changes do not automatically reload.
- Options:

- 1 Restart the Pod/Deployment.
- 2 Use checksum annotations.
- 3 Use a reloader sidecar/controller.



9. IMMUTABLE CONFIGMAPS AND SECRETS

- Mark as immutable to prevent accidental changes.
- Improves performance and stability.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  immutable: true
data:
  APP_MODE: "production"
```



10. CONFIGURATION RELOAD BEHAVIOR

CHANGE TYPE	EFFECT
ConfigMap mounted as env var	Requires Pod restart
ConfigMap mounted as file	File updated (not reloaded)
Secret mounted as env var	Requires Pod restart
Secret mounted as file	File updated (not reloaded)



11. KUBERNETES SECRET LIMITATIONS

- Stored in etcd (base64 encoded only).
- Not meant for large data (>1MB).
- Anyone with API access can read Secrets.
- Not encrypted in transit (without TLS).
- Not rotated automatically.



12. ENCRYPTION AT REST

- Enable etcd encryption for Secrets.
- Protects data at rest in etcd.
- Use KMS plugins or cloud provider key management.
- Does not replace RBAC and access control.



13. EXTERNAL SECRET-MANAGEMENT CONCEPTS

- Store Secrets outside the cluster.
- Examples: AWS Secrets Manager, HashiCorp Vault, Azure Key Vault, Google Secret Manager.
- Use controllers (e.g., External Secrets Operator, Secrets Store CSI Driver). Keeps sensitive data out of etcd.



14. CONFIGURATION SEPARATION

- Separate config per environment (dev, staging, prod).
- Avoid hardcoding values in images.
- Use overlays (Helm values, Kustomize, ConfigMaps, External Secrets).
- Follow 12-Factor App principles.



15. SECURE APPLICATION CONFIGURATION

- Use Secrets for all sensitive data.
- Apply least privilege (RBAC).
- Enable encryption at rest.
- Use TLS for all communication.
- Audit access to Secrets regularly.



16. TROUBLESHOOTING CONFIGURATION PROBLEMS

- Check Pod events: `kubectl describe pod <pod-name>`
- Verify mounts: `kubectl exec -it <pod> -- ls -l /etc/config`
- Check env vars: `kubectl exec -it <pod> -- env | grep <KEY>`
- Validate Secret/ConfigMap exists: `kubectl get secret,configmap`



SECURITY REMINDER: BASE64 ENCODING IS NOT ENCRYPTION.

ALWAYS ENABLE ENCRYPTION AT REST AND USE PROPER ACCESS CONTROLS.



— VERIQTA —

PERSISTENT STORAGE



1. VOLUMES

- Volumes provide storage that is accessible to Pods.
- Types: emptyDir, hostPath, configMap, secret, PVC, CSI, and more.
- Pod lifetime = Volume lifetime (except PVC).
- Ephemeral volumes are deleted with the Pod.



2. PERSISTENTVOLUMES (PV)

- Cluster-level storage resource.
- Provisioned by admin or dynamically by StorageClass.
- Independent of any Pod.
- Contains capacity, access modes and reclaim policy.

PV EXAMPLE

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-01
spec:
  capacity:
    storage: 10Gi
  accessModes: ["ReadWriteOnce"]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: fast-ssd
  csi:
    driver: ebs.csi.aws.com
    volumeHandle: vol-0abc123
```



3. PERSISTENTVOLUMECLAIMS (PVC)

- Pod requests storage using a PVC.
- PVCs bind to a PV that matches the request.
- Users do not consume PVs directly.
- Lives in a namespace.

PVC EXAMPLE

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
  storageClassName: fast-ssd
```



4. STORAGECLASSES

- Define storage types and provisioners.
- Used for dynamic provisioning.
- Control parameters like performance, encryption, IOPS, etc.

STORAGECLASS EXAMPLE

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```



5. DYNAMIC PROVISIONING

- PVC created → StorageClass selects provisioner.
- CSI driver creates PV automatically.
- PVC binds to the new PV.
- Works with supported storage backends.
- Simplifies storage management.



6. STATIC PROVISIONING

- Admin creates PV manually.
- PVC must match PV specifications.
- Used for pre-provisioned storage.
- Requires manual lifecycle management.



7. ACCESS MODES

MODE	DESCRIPTION
ReadWriteOnce (RWO)	Read/Write by a single node.
ReadOnlyMany (ROX)	Read-only by many nodes.
ReadWriteMany (RWX)	Read/Write by many nodes.
ReadWriteOncePod	R/W by a single Pod (alpha/stable).



8. VOLUME MODES

MODE	DESCRIPTION
Filesystem (default)	Mounted as a file system.
Block	Raw block device (no filesystem).



9. RECLAIM POLICIES

POLICY	DESCRIPTION
Retain	PV is not deleted. Manual cleanup required.
Delete	PV is deleted by the provisioner.
Recycle (Deprecated)	PV is scrubbed and made available again.



10. VOLUMEBINDINGMODE

MODE	DESCRIPTION
Immediate (default)	PV is bound immediately even before Pod is scheduled.
WaitForFirstConsumer	PV is bound only when the Pod is scheduled.



11. CSI DRIVERS

- CSI = Container Storage Interface.
- Standardizes storage integration.
- Handles provisioning, attach, mount, snapshot, resize.
- Example: ebs.csi.aws.com, pd.csi.storage.gke.io, disk.csi.azure.com.

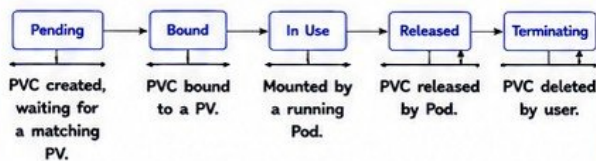


12. STATEFULSET STORAGE

- Each Pod gets its own PVC.
- PVC is named using the pattern: <claimName>-<statefulsetName>-<ordinal>
- Ensures stable identity and persistent data.
- Pods keep their data even after rescheduling.



13. PVC LIFECYCLE



14. VOLUME EXPANSION

- Supported by CSI drivers.
- Increase PVC size → PV resized.
- Filesystem expansion may be automatic (ext4, xfs).
- Requires StorageClass allowVolumeExpansion: true.



15. SNAPSHOTS

- Point-in-time copy of a volume.
- Requires CSI snapshot support.
- Useful for backup and restore.
- Can create PVC from a snapshot.



16. MOUNT FAILURES

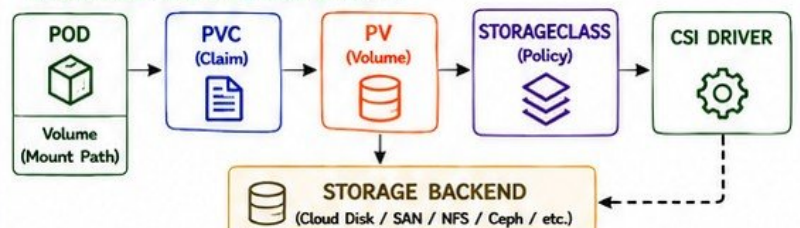
- Incorrect access mode.
- Node does not have permissions.
- Storage backend failure.
- CSI driver issues.
- Filesystem corruption.
- Network connectivity problems.



17. PENDING PVC TROUBLESHOOTING

- No matching PV available.
- StorageClass not set or invalid.
- Insufficient storage capacity.
- Access mode mismatch.
- volumeBindingMode is causing delay.
- Check events: `kubectl describe pvc <name>`
- Check StorageClass and provisioner logs.

18. STORAGE ARCHITECTURE DIAGRAM



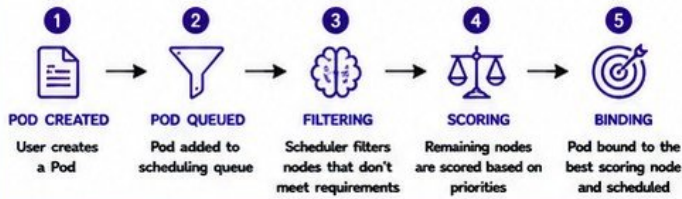
PRODUCTION NOTE: Storage failure can outlive a Pod restart, so investigate the storage layer independently from the workload.



— VERIQTA —

SCHEDULING AND NODE PLACEMENT

1. KUBERNETES SCHEDULER WORKFLOW



SCHEDULER CONSIDERS:

- ✓ Resources (CPU, Memory, Pods)
- ✓ Constraints (PDBs, node conditions, etc.)
- ✓ Policies (affinity, taints, topology, etc.)
- ✓ Custom scheduler profiles (if configured)

2. NODE SELECTOR

- Simple way to constrain Pods to nodes with matching labels.
- Key: value pair match.

```
spec:
  nodeSelector:
    disktype: ssd
    environment: production
```

3. NODE AFFINITY

- More powerful than nodeSelector.
- Supports requirements and preferences.
- Types: requiredDuringSchedulingIgnoredDuringExecution, preferredDuringSchedulingIgnoredDuringExecution

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values: ["ssd"]
```

4. POD AFFINITY

- Attract Pods to nodes where certain Pods are already running.
- Types: required or preferred.

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: backend
            topologyKey: kubernetes.io/hostname
```

5. POD ANTI-AFFINITY

- Prevent Pods from running on nodes where certain Pods exist.
- Great for spreading workloads.

```
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: frontend
            topologyKey: kubernetes.io/hostname
```

6. TAINTS

- Mark nodes so that unwanted Pods are not scheduled.
- Pods must have matching tolerations.

NODE (TAINED)

key: dedicated
value: gpu
effect: NoSchedule

7. TOLERATIONS

- Allow Pods to be scheduled on tainted nodes.
- Must match taint key, value and effect.

```
spec:
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "gpu"
      effect: "NoSchedule"
```

8. TOPOLOGY SPREAD CONSTRAINTS

- Spread Pods evenly across topology domains (zone, hostname, etc.).
- Prevents single points of failure.

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: myapp
```

9. NODE LABELS

- Key-value pairs attached to nodes.
- Used by selectors, affinity, and other features.

EXAMPLES:

```
kubernetes.io/hostname: node-1
topology.kubernetes.io/zone: us-east-1a
disktype: ssd
environment: production
```

10. RESOURCE-BASED SCHEDULING

- Scheduler ensures nodes have enough:
 - ✓ CPU
 - ✓ Memory
 - ✓ Ephemeral Storage
 - ✓ Pods
- Based on requests, not limits.



11. POD PRIORITY

- Assign important Pods a higher priority.
- Higher priority Pods get scheduled first.

priorityClassName:
high-priority



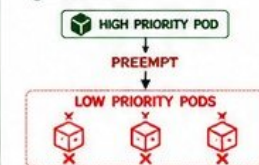
12. PRIORITYCLASSES

- Define global priority levels.
- Used by Pods via priorityClassName.
- Higher value = higher priority.

EXAMPLE PRIORITYCLASSES		
NAME	VALUE	DESCRIPTION
system-cluster-critical	2000000000	Critical system Pods
system-node-critical	1000000000	Node critical Pods
high-priority	100000	Business critical Pods
normal	0	Default priority
low-priority	-1000	Low importance Pods

13. PREEMPTION

- If a node lacks resources, lower priority Pods may be evicted to schedule higher priority Pods.
- Helps important workloads get resources.



14. POD DISRUPTION BUDGETS (PDB)

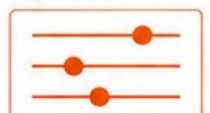
- Limits the number of Pods that can be disrupted at once.
- Protects availability during voluntary disruptions.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: myapp
```



15. SCHEDULING PROFILES (CONCEPT)

- Allows multiple scheduler configurations in one cluster.
- Each profile can have different plugins and policies.
- Useful for multi-tenant or specialized workloads.



16. UNSCHEDULABLE PODS

- Pod remains in Pending state.
- Reasons:
 - ✓ Insufficient resources
 - ✓ Node selectors mismatch
 - ✓ Taints not tolerated
 - ✓ Affinity rules not satisfied
 - ✓ PDB constraints

Check events and descriptions for details.

17. FAILED SCHEDULING (EVENT)

- Look for "FailedScheduling" events.
- Use kubectl describe pod <pod>
- Events give the exact reason for scheduling failure.

```
Warning FailedScheduling
0/5 nodes are available:
2 node(s) didn't match node selector,
1 Insufficient memory,
2 node(s) had untolerated taint.
```

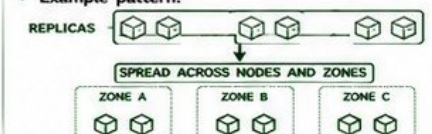
18. MULTI-ZONE PLACEMENT

- Spread workloads across zones for high availability.
- Use topology spread constraints and anti-affinity.
- Avoid zone as single point of failure.



19. HIGH-AVAILABILITY PLACEMENT PATTERNS

- Use anti-affinity to avoid co-locating replicas on the same node.
- Use topology spread constraints across zones.
- Combine PDB + Anti-affinity + Multiple zones.
- Example pattern:



SENIOR ENGINEER TIP:

DESIGN PLACEMENT RULES FOR FAILURE DOMAINS, NOT SIMPLY FOR INDIVIDUAL NODES.



VERIQTA

[youtube.com/@veriqta](https://www.youtube.com/@veriqta)
github.com/veriqta
[linkedin.com/in/veriqta](https://www.linkedin.com/in/veriqta)
[instagram.com/veriqta](https://www.instagram.com/veriqta)
twitter.com/veriqta
t.me/veriqta

— VERIQTA —

RESOURCE MANAGEMENT, QOS AND EVICTIONS



1. CPU REQUESTS

- Minimum CPU guaranteed to the container.
- Scheduler uses it to place Pods on nodes.
- Unit: millicores (m)
- 1000m = 1 CPU

```
resources:
  requests:
    cpu: "500m" # 0.5 CPU
```



2. CPU LIMITS

- Maximum CPU the container can use.
- Prevents a container from consuming too much CPU.
- Causes throttling when limit is reached.

```
resources:
  limits:
    cpu: "1000m" # 1 CPU
```



3. MEMORY REQUESTS

- Minimum memory guaranteed to the container.
- Scheduler uses it for placement.
- Helps avoid OOMKills.

```
resources:
  requests:
    memory: "512Mi"
```



4. MEMORY LIMITS

- Maximum memory allowed.
- If usage exceeds the limit, the container is OOMKilled.
- Unit: Mi, Gi

```
resources:
  limits:
    memory: "1024Mi"
```



5. CPU THROTTLING

- Happens when container hits CPU limit.
- CPU usage is capped by CFS (Completely Fair Scheduler).
- Symptoms: high CPU usage not reflected in performance.
- Check with: `kubectl top pod`



6. OOMKILLED

- Happens when container uses more memory than its limit.
- Kernel kills the process.
- Pod restarts (may lead to CrashLoopBackOff).
- Check with: `kubectl describe pod`



7. QOS CLASSES

- Kubernetes assigns QoS class to each Pod.
- Determines eviction priority and resource guarantees.
- Check with: `kubectl describe pod | grep QoS`

8. QOS CLASSES EXPLAINED

CLASS	CRITERIA	CPU REQUEST/LIMIT	MEMORY REQUEST/LIMIT	EVICTON PRIORITY	GUARANTEE
GUARANTEED	Requests == Limits for CPU and Memory	Equal	Equal	Lowest	Highest Stability
BURSTABLE	Requests < Limits for CPU or Memory	Not Equal	Not Equal	Medium	Best Effort But Not Guaranteed
BESTEFFORT	No Requests No Limits	None	None	Highest	No Guarantee

9. QOS BEHAVIOR

- Kuberanteed Pods are evicted last.
- BestEffort Pods are evicted first.
- Burstable Pods are in between.
- Node pressure triggers evictions.



10. NODE ALLOCATABLE RESOURCES

- Total resources - system reservations = allocatable.
- Kubelet reserves resources for system daemons.
- View with: `kubectl describe node`



11. RESOURCEQUOTA

- Limits total resources that can be used in a namespace.
- Prevents a single team from consuming everything.
- Example:

```
apiVersion: v1
kind: ResourceQuota
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
```



12. LIMITRANGE

- Sets default or min/max limits for resource requests/limits.
- Ensures Pods don't run with unreasonable values.
- Example:

```
apiVersion: v1
kind: LimitRange
spec:
  limits:
    - type: Container
      defaultRequest:
        cpu: "200m"
        memory: "256Mi"
```



13. NODE PRESSURE

- Node pressure leads to evictions.
- Kubelet monitors resource pressure.
- Types:
 - Memory Pressure - Memory low
 - Disk Pressure - Disk space low
 - PID Pressure - Too many PIDs
- Check with: `kubectl describe node`



14. EVICTIONS

- Kubelet evicts Pods to reclaim resources.
- Eviction order (from first to last):
 1. BestEffort Pods
 2. Burstable Pods
 3. Guaranteed Pods
- Eviction is based on QoS + resource usage.
- Graceful termination period applies.



15. EVICTION SIGNALS

- MemoryPressure
- DiskPressure
- PIDPressure
- Check signals: `kubectl describe node | grep -A2 "Conditions"`



16. RESOURCE TROUBLESHOOTING

1. Check Pod status and events `kubectl describe pod`
2. Check node pressure `kubectl describe node`
3. Check resource usage `kubectl top pod` `kubectl top node`
4. Check QoS class `kubectl describe pod | grep QoS`
5. Check metrics in Grafana/Prometheus



17. RIGHT-SIZING PRINCIPLES

- Start with requests based on actual usage.
- Set limits 1.5x - 2x of requests.
- Avoid equal request = limit unless needed.
- Monitor, adjust, repeat.
- Don't guess. Measure.
- Right-size for performance + cost.

18. RESOURCE IMPACT SUMMARY

SCENARIO	CPU IMPACT	MEMORY IMPACT	RESULT
No CPU Limit	Pod can use all available CPU	N/A	Good for batch jobs, risk noisy neighbors
CPU Limit Hit	Throttling occurs	N/A	Slower performance
No Memory Limit	Pod can use all node memory	Risk of node OOM	Other Pods may be evicted
Memory Limit Hit	N/A	Container killed	OOMKilled

19. COMMON ISSUES

ISSUE	CAUSE	SOLUTION
Pod Pending	Insufficient resources	Increase capacity or reduce requests
Pod Evicted	Node pressure	Check node, reduce usage
CrashLoopBackOff	OOMKilled	Increase memory limit
High CPU Throttling	CPU limit too low	Increase CPU limit or optimize app
Unstable Performance	Wrong requests/limits	Right-size using metrics



20. QUICK CHECK COMMANDS

```
kubectl top pod
kubectl top node
kubectl describe pod <pod-name>
kubectl describe node <node-name>
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get pods --field-selector=status.phase=Failed
```



PERFORMANCE TIP: CPU LIMITS AND MEMORY LIMITS BEHAVE DIFFERENTLY. DO NOT TROUBLESHOOT THEM AS IF THEY WERE INTERCHANGEABLE.



VERIQTA



youtube.com/@veriqta



github.com/veriqta



linkedin.com/in/veriqta



instagram.com/veriqta



twitter.com/veriqta



t.me/veriqta

AUTOSCALING AND CAPACITY



1. HORIZONTAL POD AUTOSCALER (HPA)

- Automatically scales the number of Pod replicas.
- Works with Deployments, StatefulSets and ReplicaSets.
- Based on metrics (CPU, Memory, Custom, External).
- Ensures application availability and performance.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```



2. VERTICAL POD AUTOSCALER (VPA) CONCEPTS

- Adjusts CPU/Memory requests and limits for Pods.
- Does NOT change the number of replicas.
- Can update Pods automatically (in-place restart) or in recommendation mode.
- Helps right-size containers.

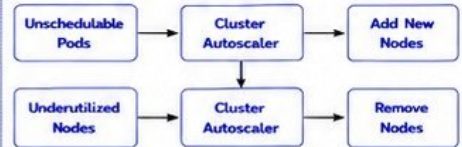
```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  updatePolicy:
    updateMode: "Auto" # Off | Initial | Auto
  resourcePolicy:
    containerPolicies:
    - containerName: web
      minAllowed:
        cpu: "100m"
        memory: "128Mi"
      maxAllowed:
        cpu: "2"
        memory: "2Gi"
```



3. NODE AUTOSCALING CONCEPTS (CLUSTER AUTOSCALER)

- Automatically adjusts the number of nodes in the cluster.
- Adds nodes when Pods are unschedulable.
- Removes unused nodes to save cost.
- Works with cloud provider APIs.

CLUSTER AUTOSCALER FLOW



SUPPORTED SCALING ACTIONS



4. METRICS SERVER

- Collects resource metrics from Kubelets.
- Required for HPA, VPA and other metrics.
- Lightweight and efficient.

```
# Install Metrics Server
kubectl apply -f \
  https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```



5. CPU-BASED SCALING

- Scales Pods based on CPU utilization.
- Most common autoscaling metric.
- Uses Metrics Server data.

Target Example:
Average CPU Utilization
> 70% -> Scale Up
< 70% -> Scale Down



6. MEMORY-BASED SCALING

- Scales Pods based on Memory utilization.
- Prevents memory pressure and OOMKilled.
- Requires Metrics Server.

Target Example:
Average Memory Utilization
> 80% -> Scale Up
< 80% -> Scale Down



7. CUSTOM METRICS

- Application-specific metrics.
- Exposed via Prometheus Adapter.
- Example: requests per second, queue length.

Example Metric:
http_requests_per_second
Target: Average Value 100



8. EXTERNAL METRICS

- Metrics from outside the cluster.
- Example: SQS queue length, CloudWatch metrics, etc.

Example Source:
AWS SQS Queue Messages Visible
Target: > 100



9. HPA BEHAVIOR

- Define how HPA reacts to metric changes.
- Control scale-up and scale-down behavior.
- Prevents aggressive or unstable scaling.

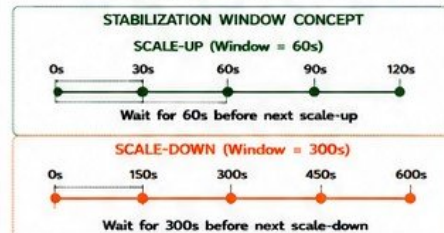
Behavior Example (HPA v2)

```
behavior:
  stabilizationWindowSeconds: 60
  policies:
  - type: Percent
    value: 100
    periodSeconds: 15
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
    - type: Percent
      value: 50
      periodSeconds: 60
```



10. STABILIZATION WINDOWS

- Scale-up window: wait before scaling up again.
- Scale-down window: wait before scaling down.
- Reduces flapping and unnecessary scaling.



11. SCALING POLICIES

- Define the rules for scaling actions.
- Types: Pods | Percent
- Prevents scaling too fast.
- Works with stabilization windows.

POLICY EXAMPLE

TYPE	VALUE	PERIOD	MEANING
Pods	4	60s	Add/Remove up to 4 Pods per 60s
Percent	100	60s	Add/Remove up to 100%, per 60s



12. RESOURCE REQUESTS AND HPA

- HPA uses actual usage, but scheduling uses requests.
- Set appropriate requests for stable scaling.
- Too low requests = poor performance.
- Too high requests = inefficient scaling.

Best Practice:
Set requests ~50-70% of expected usage



13. CAPACITY PLANNING

- Understand growth patterns.
- Plan for peak load.
- Consider cluster, node and application limits.
- Factor in cost, availability and failure domains.

Key Questions:

- What is the expected growth?
- What is the max capacity needed?
- What are the dependencies?



14. SCALING BOTTLENECKS

- Database connections
- External APIs / Rate limits
- Load balancer limits
- Disk I/O or throughput
- Network bandwidth
- Stateful dependencies

Tip: Identify and fix the true bottleneck before adding more replicas.



15. COLD STARTS

- New Pods take time to start.
- Large images increase startup time.
- Databases and caches may warm up slowly.
- Use readiness probes to avoid sending traffic too early.

Minimize Cold Starts:

- Use smaller images
- Enable image caching
- Keep app lightweight



16. SCALE-UP AND SCALE-DOWN BEHAVIOR

SCALE-UP

- Metric exceeds target
- HPA calculates desired replicas
- New Pods are created
- Pods become Ready
- Traffic is served

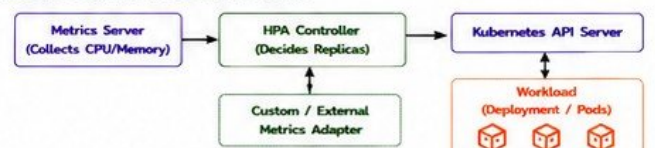


SCALE-DOWN

- Metric drops below target
- HPA marks excess Pods
- Pods are terminated gracefully
- Connections drain
- Capacity is reduced



17. AUTOSCALING ARCHITECTURE



SCALING NOTE:

AUTOSCALING CANNOT COMPENSATE FOR AN ARCHITECTURE WHOSE TRUE BOTTLENECK EXISTS OUTSIDE THE SCALED WORKLOAD.



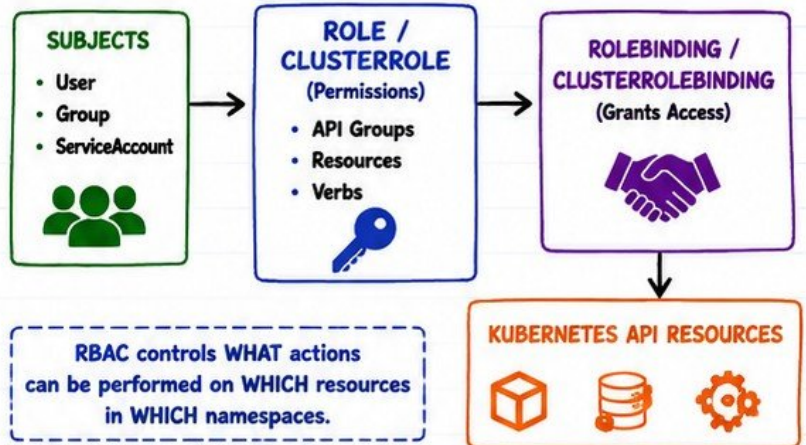
14. KUBERNETES SECURITY AND RBAC

1. CORE SECURITY CONCEPTS

- AUTHENTICATION
- AUTHORIZATION
- SERVICEACCOUNTS
- ROLES
- CLUSTERROLES
- ROLEBINDINGS
- CLUSTERROLEBINDINGS
- RBAC INHERITANCE
- LEAST PRIVILEGE
- KUBECTL AUTH CAN-I



2. RBAC OVERVIEW



3. POD SECURITY STANDARDS

PRIVILEGED	BASELINE	RESTRICTED
Unrestricted. Not recommended.	Minimally restrictive. Prevents known privilege escalations.	Heavily restricted. Strong security for production.

4. SECURITYCONTEXT ESSENTIALS

- RUNASNONROOT: true
- RUNASUSER: <non-root-uid>
- ALLOWPRIVILEGEESCALATION: false
- READONLYROOTFILESYSTEM: true
- CAPABILITIES: drop all, add only what is needed
- SECCOMPPROFILE: RuntimeDefault



5. LINUX CAPABILITIES

CAPABILITY	DESCRIPTION
CHOWN	Change file ownership
NET_BIND_SERVICE	Bind to privileged ports (<1024)
SYS_TIME	Set system time
DAC_OVERRIDE	Bypass file read/write/execute permissions
SYS_ADMIN	Powerful capability, avoid granting
NET_ADMIN	Network configuration

6. SECURITY CONTROLS



SECCOMP

Restricts system calls available to containers.



READ-ONLY ROOT FS

Prevents writing to the container's root filesystem.



PRIVILEGED CONTAINERS

Avoid running containers in privileged mode.



RESOURCE LIMITS

Prevent resource abuse and noisy neighbors.

7. ADMISSION CONTROL (CONCEPTS)

- ✓ VALIDATING ADMISSION: Validates requests before storing.
- ✓ MUTATING ADMISSION: Modifies requests before storing.
- ✓ USE POLICIES TO ENFORCE SECURITY STANDARDS.
- ✓ EXAMPLES: OPA GATEKEEPER, KYVERNO, PSA, CUSTOM WEBHOOKS.

VERIQTA

8. AUDIT LOGGING



- RECORDS API REQUESTS AND RESPONSES.
- ESSENTIAL FOR COMPLIANCE AND FORENSICS.
- CHECK WHO DID WHAT, WHEN, AND FROM WHERE.
- ENABLE AUDIT LOGGING ON THE API SERVER.

VERIQTA

9. SECRETS SECURITY

- STORE SENSITIVE DATA IN SECRETS, NOT IN IMAGES.
- ENABLE ENCRYPTION AT REST.
- USE RBAC TO LIMIT SECRET ACCESS.
- AVOID EXPOSING SECRETS IN ENV VARS OR LOGS.



VERIQTA

10. NODE SECURITY



- KEEP NODES AND RUNTIME UP TO DATE.
- USE STRONG SSH ACCESS CONTROLS.
- ENABLE FIREWALL AND NETWORK SEGMENTATION.
- MONITOR NODES FOR SUSPICIOUS ACTIVITY.

VERIQTA



SECURITY REMINDER: CLUSTER-ADMIN ACCESS SHOULD BE EXCEPTIONAL, CONTROLLED, AND AUDITABLE.



VERIQTA

CONNECT WITH VERIQTA



youtube.com/@veriqta



github.com/veriqta



linkedin.com/in/veriqta



twitter.com/veriqta



instagram.com/veriqta



t.me/veriqta

15. OBSERVABILITY, LOGGING AND MONITORING

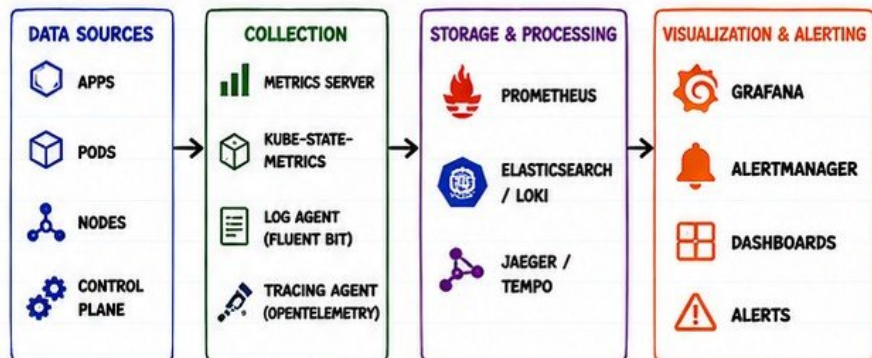
1. OBSERVABILITY PILLARS

- KUBERNETES EVENTS
- POD LOGS
- PREVIOUS CONTAINER LOGS
- METRICS
- METRICS SERVER
- PROMETHEUS CONCEPTS
- GRAFANA CONCEPTS
- APPLICATION METRICS
- NODE METRICS
- KUBE-STATE-METRICS
- CONTROL-PLANE MONITORING
- DISTRIBUTED TRACING CONCEPTS
- STRUCTURED LOGGING
- CENTRALIZED LOGGING
- ALERTING
- GOLDEN SIGNALS
- SLI, SLO AND SLA CONCEPTS
- USEFUL OBSERVABILITY COMMANDS
- MONITORING ARCHITECTURE



OBSERVABILITY HELPS YOU UNDERSTAND THE HEALTH, PERFORMANCE AND BEHAVIOR OF YOUR APPLICATIONS AND CLUSTER.

2. OBSERVABILITY STACK OVERVIEW



COLLECT → STORE → VISUALIZE → ALERT → ACT
THE OBSERVABILITY FLOW

3. GOLDEN SIGNALS



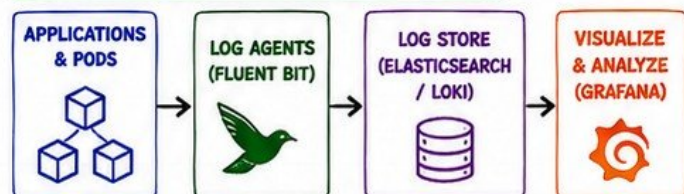
5. USEFUL OBSERVABILITY COMMANDS

<pre> > kubectl get events -A --sort-by=.lastTimestamp kubectl describe pod <pod> kubectl logs <pod> kubectl logs <pod> -c <container> kubectl logs <pod> -c <container> --previous kubectl top pod kubectl top node kubectl top pods -A kubectl get --raw /metrics kubectl auth can-i --list kubectl get componentstatuses </pre>	- List recent events - See events and status - View pod logs - View container logs - Previous container logs - Resource usage (pods) - Resource usage (nodes) - All namespaces - View raw metrics from API server - Check permissions - Control plane component health
---	--

4. METRICS TYPES

TYPE	DESCRIPTION	EXAMPLES
APPLICATION METRICS	Business and app specific metrics from the app.	HTTP requests, orders, signups
NODE METRICS	Metrics about nodes and infrastructure.	CPU, Memory, Disk, Network
KUBERNETES METRICS	K8s objects and component metrics.	Pods, Deployments, API server, etc.
CONTROL PLANE METRICS	Health and performance of control plane components.	API server latency, etcd size

6. LOGGING ARCHITECTURE EXAMPLE



ALWAYS LOG IN STRUCTURED FORMAT (JSON) TO MAKE SEARCHING AND ALERTING EFFECTIVE.

7. ALERTING FLOW



8. SLI, SLO AND SLA OVERVIEW

SLI	SERVICE LEVEL INDICATOR	A measurable metric that indicates how well a service is doing.	e.g. 99.9% of requests succeed
SLO	SERVICE LEVEL OBJECTIVE	A target value for the SLI over a given time period.	e.g. 99.9% success rate per month
SLA	SERVICE LEVEL AGREEMENT	A commitment to users, often backed by a contract.	e.g. 99.9% uptime guaranteed



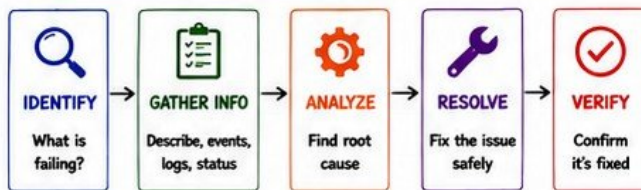
PRODUCTION NOTE: LOGS TELL YOU WHAT HAPPENED. METRICS SHOW PATTERNS. TRACES SHOW HOW A REQUEST MOVED THROUGH THE SYSTEM.



CONNECT WITH VERIQTA

16. KUBERNETES TROUBLESHOOTING PLAYBOOK

1. TROUBLESHOOTING WORKFLOW



ALWAYS START WITH OBSERVING THE CURRENT STATE BEFORE MAKING ANY CHANGES.

3. ESSENTIAL TROUBLESHOOTING COMMANDS

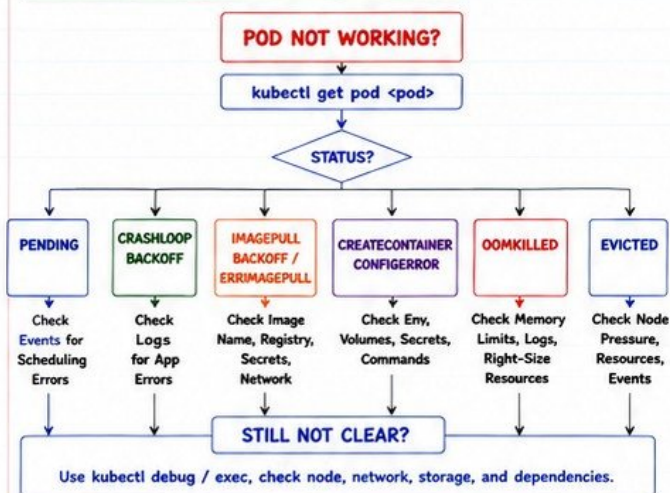
COMMAND	PURPOSE
<code>kubectl get pods -A</code>	List all pods in all namespaces
<code>kubectl describe pod <pod> -n <ns></code>	Detailed pod status, events, conditions
<code>kubectl get events -A --sort-by=.metadata.creationTimestamp</code>	Recent events across the cluster
<code>kubectl logs <pod> -n <ns></code>	Current container logs
<code>kubectl logs <pod> -n <ns> -c <container></code>	Logs for a specific container
<code>kubectl logs <pod> -n <ns> -p</code>	Previous container logs
<code>kubectl exec -it <pod> -n <ns> -- /bin/sh</code>	Open shell in running container
<code>kubectl top pod -n <ns></code>	Resource usage for pods
<code>kubectl top node</code>	Resource usage for nodes
<code>kubectl get nodes</code>	Check node status
<code>kubectl describe node <node></code>	Detailed node info and events

5. INFORMATION HIERARCHY (CHECK IN THIS ORDER)



THIS ORDER SAVES TIME AND REVEALS THE ROOT CAUSE FASTER.

6. FAILURE-ISOLATION DECISION TREE



2. COMMON POD FAILURE STATES

STATE	MEANING	WHAT TO CHECK
• PENDING	Pod not yet scheduled	Events, scheduling, resources
• CRASHLOOPBACKOFF	Container crashes repeatedly	Logs, probes, code, config
• IMAGEPULLBACKOFF	Image pull failed	Image name, registry, secrets
• ERRIMAGEPULL	Cannot pull image	Registry, auth, network
• CREATECONTAINERCONFIGERROR	Invalid container config	Env, volume, command, secret
• OOMKILLED	Killed due to memory	Memory limits, usage, code
• EVICTED	Evicted by kubelet	Resources, node pressure
• CONTAINERCREATING	Creating container	Volume, image, CNI, secret
• FAILED SCHEDULING	Scheduler cannot place pod	Resources, taints, affinity
• PROBE FAILURES	Liveness/Readiness failed	Probe config, app health
• DNS FAILURES	Cannot resolve DNS	CoreDNS, network, policy
• SERVICE CONNECTIVITY FAILURES	Cannot reach service	Service, endpoints, network
• INGRESS FAILURES	Ingress not working	Ingress rules, controller, svc
• PVC FAILURES	Persistent volume issues	PVC, PV, StorageClass, events
• NODE FAILURES	Node not healthy	Node status, kubelet, events

4. DEBUGGING TOOLS



KUBECTL DEBUG

- Launch a debug container to troubleshoot running pods.
`kubectl debug -it <pod> -n <ns> --image=busybox --target=<container> -- /bin/sh`
- Copy files for debugging.
`kubectl cp <pod>:/path/file ./file -n <ns>`



EPHEMERAL DEBUGGING CONTAINERS

- Add temporary containers without restarting the pod.
`kubectl debug -it <pod> -n <ns> --image=nicolaka/netshoot --target=<container> -- /bin/sh`
- Great for network, DNS and connectivity debugging.

7. COMMON ROOT CAUSE AREAS & WHAT TO CHECK

APPLICATION	Logs, errors, config, probes, dependencies
IMAGE	Image name, tag, registry, secrets
RESOURCES	CPU, Memory, Requests, Limits, Quotas
SCHEDULING	Taints, Affinity, Node selectors, Resources
NETWORK	CNI, DNS, NetworkPolicy, Services, Ingress
STORAGE	PVC, PV, StorageClass, Mounts, Permissions
NODE	Node status, Kubelet, Disk, Memory, Pressure
EXTERNAL DEPENDENCIES	DB, APIs, Queues, Third-party services

8. USEFUL SHORTCUTS

- ☒ `kubectl describe pod <pod> -n <ns> --show-events`
- ☒ `kubectl get events -A --sort-by=.lastTimestamp`
- ☒ `kubectl logs <pod> -n <ns> -c <container> --previous`
- ☒ `kubectl debug -it <pod> -n <ns> --image=busybox --target=<container> -- /bin/sh`



DEBUGGING INSIGHT:
START WITH STATUS, CONDITIONS AND EVENTS
BEFORE CHANGING CONFIGURATION.



CONNECT WITH VERIQTA



youtube.com/@veriqta



github.com/veriqta



linkedin.com/in/veriqta



twitter.com/veriqta



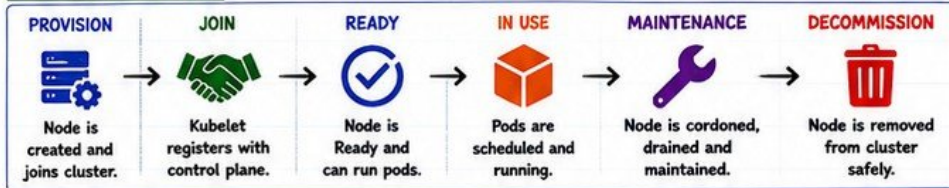
instagram.com/veriqta



t.me/veriqta

17. NODES, CLUSTER OPERATIONS AND MAINTENANCE

1. NODE LIFECYCLE



2. NODE CONDITIONS

CONDITION	STATUS	MEANING
READY	TRUE	Node is healthy and accepting pods.
	FALSE	Node is not healthy or unreachable.
MEMORY PRESSURE	TRUE	Node is low on memory.
	FALSE	Sufficient memory available.
DISK PRESSURE	TRUE	Low disk space on node.
	FALSE	Sufficient disk space available.
PID PRESSURE	TRUE	Not enough PID resources.
	FALSE	PID resources are available.
NETWORK UNAVAILABLE	TRUE	Node network is unhealthy.
	FALSE	Node network is healthy.

3. ESSENTIAL NODE OPERATIONS

	CORDON NODE Mark node unschedulable.	<code>kubectl cordon <node-name></code>
	DRAIN NODE Evict pods and prepare node for maintenance.	<code>kubectl drain <node-name> \</code> <code>--ignore-daemonsets \</code> <code>--delete-emptydir-data</code>
	UNCORDON NODE Make node schedulable again.	<code>kubectl uncordon <node-name></code>
	CHECK NODE STATUS View node conditions.	<code>kubectl get nodes</code>
	DESCRIBE NODE Detailed node information.	<code>kubectl describe node <node-name></code>

4. KUBELET HEALTH



Kubelet is the agent that runs on each node and ensures containers are running.

CHECK KUBELET STATUS

- `systemctl status kubelet`
- `journalctl -u kubelet -f`
- `kubectl get nodes`
- `kubectl describe node <node-name>`

5. NODE CAPACITY AND ALLOCATABLE

RESOURCE	CAPACITY	ALLOCATABLE
CPU (CORES)	16	14
MEMORY (Gi)	64	58
PODS	110	100
EPHEMERAL STORAGE	100Gi	85Gi

CAPACITY
• Total resources on the node.

ALLOCATABLE
• Resources available for pods after system reservations.

`kubectl describe node <node-name> | grep -A 10 Allocatable`

6. SAFE MAINTENANCE FLOW

- CORDON THE NODE**
`kubectl cordon <node-name>`
- DRAIN THE NODE**
`kubectl drain <node-name> \`
`--ignore-daemonsets --delete-emptydir-data`
- PERFORM MAINTENANCE**
(Patch, OS update, reboot, etc.)
- UNCORDON THE NODE**
`kubectl uncordon <node-name>`
- VERIFY NODE IS READY**
`kubectl get nodes`

7. POD EVICTION DURING DRAINS

- Pods are evicted respecting PDBs.
- Pods with no PDB may be evicted.
- DaemonSet pods are ignored by default.
- Local storage pods may not be rescheduled.



8. DAEMONSET BEHAVIOR

DaemonSet pods are not evicted during drain by default. They run on the node even when it is unschedulable.



9. PDB IMPACT

- PDBs control how many pods can be unavailable.
- If PDB blocks eviction, drain will wait or fail.

10. NODE UPGRADES

- Cordon node
- Drain node
- Upgrade OS / Kernel / Container Runtime
- Reboot node
- Verify kubelet is healthy
- Uncordon node
- Verify node becomes Ready

UPGRADE ORDER

Worker Nodes First
(One by one)
↓
Control Plane Nodes
(One by one)

11. CONTROL-PLANE CONSIDERATIONS



- Maintain quorum ($\geq 2/3$ control-plane nodes).
- Upgrade control-plane components one at a time.
- Check etcd health before upgrades.
- Verify API server, scheduler, controller-manager.
- Ensure backups before upgrading.

14. VERSION SKEW AWARENESS

COMPONENT	SUPPORTED SKEW
kube-apiserver	N / A (Target Version)
kubelet	-3 minor versions
kube-proxy	-3 minor versions
kubectl	+/- 1 minor version

15. MAINTENANCE CHECKLIST

- | | |
|---|--|
| ☒ Backup etcd | ☒ Perform maintenance |
| ☒ Verify node health | ☒ Verify kubelet and runtime |
| ☒ Check key workloads | ☒ Uncordon node |
| ☒ Check PDBs | ☒ Verify node is Ready |
| ☒ Cordon node | ☒ Monitor workloads |
| ☒ Drain node | ☒ Document changes |

13. CLUSTER UPGRADE PLANNING

- Review release notes.
- Check deprecated APIs.
- Backup etcd.
- Upgrade worker nodes first.
- Upgrade control-plane nodes.
- Validate workloads.
- Monitor after upgrade.



USEFUL COMMANDS

- `kubectl get nodes`
- `kubectl describe node <node>`
- `kubectl cordon <node>`
- `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`
- `kubectl uncordon <node>`
- `kubectl get events --sort-by=.lastTimestamp`



ENTERPRISE ADVICE:
NODE MAINTENANCE IS AN APPLICATION-AVAILABILITY EVENT,
NOT MERELY AN INFRASTRUCTURE TASK.



CONNECT WITH VERIQTA

18. HELM, KUSTOMIZE, GITOPS AND DELIVERY

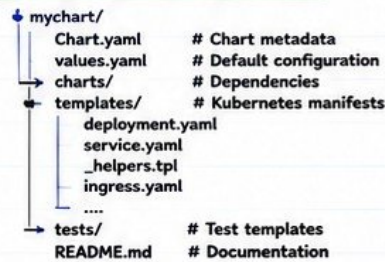
1. HELM CHARTS OVERVIEW

- Package Kubernetes resources for easy deployment.
- Reusable, shareable and versioned.
- Templated with Go template engine.
- Managed as releases.



VERIQTA

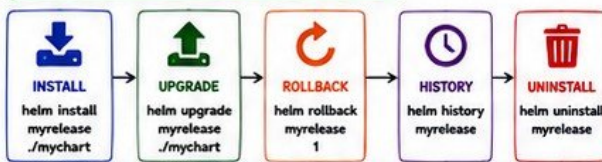
2. HELM CHART STRUCTURE



KEY FILES

Chart.yaml	Chart info, version, dependencies
values.yaml	Default configuration values
templates/	Kubernetes manifests with templates
_helpers.tpl	Reusable template functions
charts/	Subcharts and dependencies

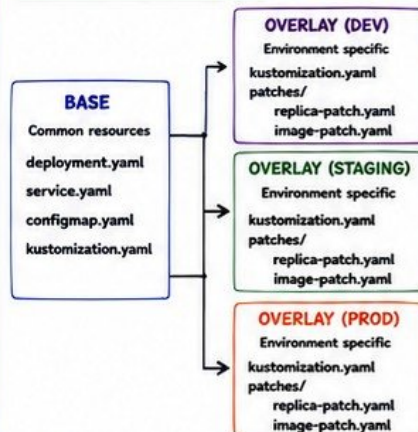
3. HELM WORKFLOW & COMMANDS



COMMON HELM COMMANDS

helm install myrelease ./mychart -f values.yaml	Install a chart
helm upgrade myrelease ./mychart -f values.yaml	Upgrade a release
helm rollback myrelease 1	Rollback to revision 1
helm history myrelease	Show release history
helm status myrelease	Show release status
helm list --all-namespaces	List all releases
helm uninstall myrelease	Uninstall a release
helm template mychart	Render templates locally
helm lint mychart	Validate chart

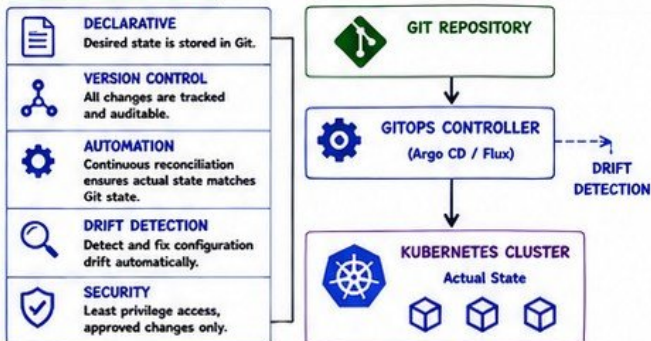
4. KUSTOMIZE OVERVIEW



KUSTOMIZE COMPONENTS

- Bases: Reusable common configuration
- Overlays: Environment specific customization
- Patches: Strategic merge or JSON6902
- Transformers: Modify resources
- Generators: ConfigMap, Secret, etc.
- Built-in: No template engine, simple and reliable

5. GITOPS PRINCIPLES



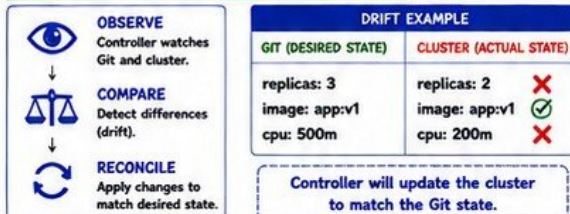
KUSTOMIZE COMMANDS

- kubectl apply -k ./overlays/prod
- kubectl kustomize ./overlays/prod
- kubectl diff -k ./overlays/prod

6. ARGO CD vs FLUX CONCEPTS

	ARGO CD	FLUX
Architecture	Pull-based	Pull-based
CD Tool	Yes (Built-in UI)	Yes (CLI / UI via extensions)
Configuration	Declarative (Git)	Declarative (Git)
Kubernetes CRD	Extensive	Lightweight
Multi-tenancy	Project based	Namespace based
Image Update	Argo Rollouts / Image Updater	Image Automation
Best For	Enterprise CD with rich UI	GitOps-native Kubernetes principles

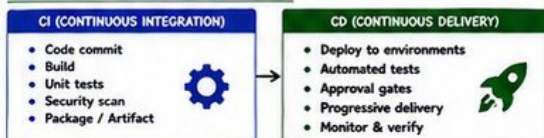
7. DRIFT DETECTION & RECONCILIATION



DRIFT EXAMPLE	
GIT (DESIRED STATE)	CLUSTER (ACTUAL STATE)
replicas: 3	replicas: 2
image: app:v1	image: app:v1
cpu: 500m	cpu: 200m

Controller will update the cluster to match the Git state.

9. CI vs CD RESPONSIBILITIES



8. PROGRESSIVE DELIVERY CONCEPTS



10. PRODUCTION DEPLOYMENT WORKFLOW



PRODUCTION NOTE: GITOPS WORKS BEST WHEN PRODUCTION STATE CHANGES THROUGH A CONTROLLED RECONCILIATION PATH RATHER THAN MANUAL CLUSTER EDITS.

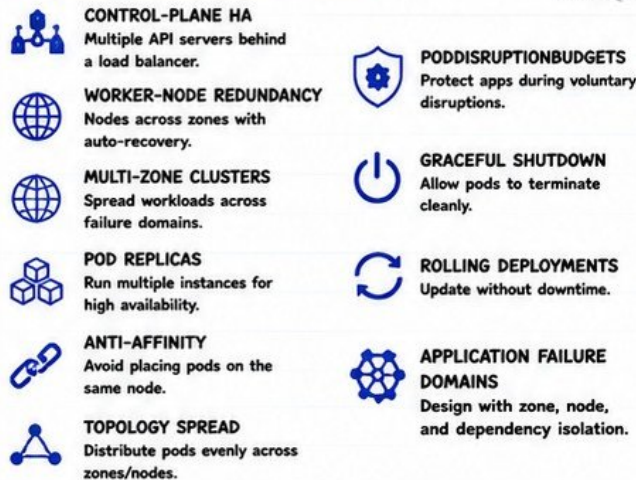


CONNECT WITH VERIQTA

19. HIGH AVAILABILITY, RELIABILITY AND DISASTER RECOVERY

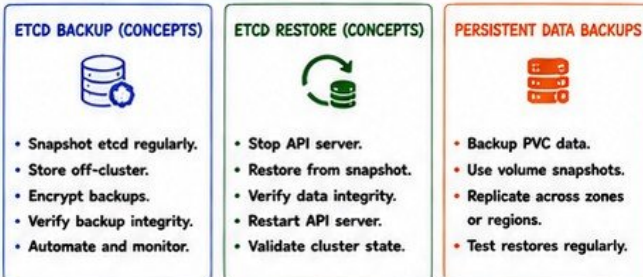
1. RELIABILITY BUILDING BLOCKS

VERIQTA

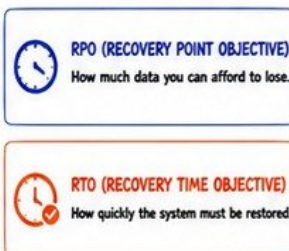


3. DATA PROTECTION: BACKUPS & RESTORE

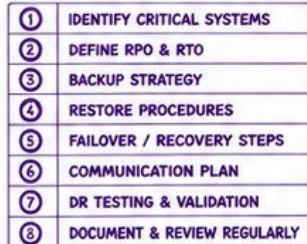
VERIQTA



4. RPO & RTO EXPLAINED



5. DISASTER RECOVERY PLAN



8. GRACEFUL SHUTDOWN FLOW

VERIQTA

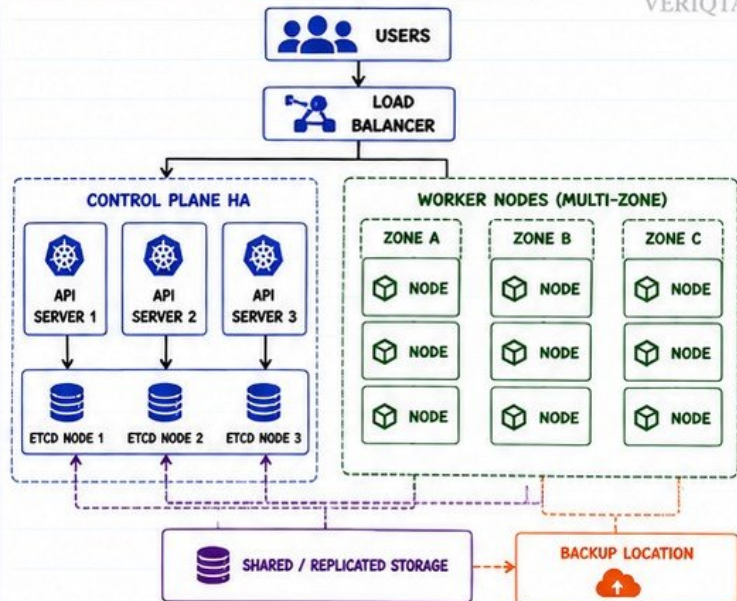


9. ROLLING DEPLOYMENTS (ZERO DOWNTIME)



2. HIGH AVAILABILITY ARCHITECTURE DIAGRAM

VERIQTA



6. TESTING, VERIFICATION & FAILURE SIMULATION

VERIQTA



7. HIGH AVAILABILITY MAINTENANCE CHECKLIST

VERIQTA

- | | |
|---|--|
| ☒ Control-plane nodes are healthy | ☒ RPO & RTO documented |
| ☒ etcd backups are up to date | ☒ DR plan reviewed |
| ☒ Worker nodes are Ready | ☒ Certificate expiry checked |
| ☒ Pod replicas are healthy | ☒ Cluster upgrade plan ready |
| ☒ PDBs are configured | ☒ Monitoring & alerting active |
| ☒ Anti-affinity / Topology spread set | ☒ Failure simulations done |
| ☒ Storage snapshots verified | ☒ Runbooks are updated |
| ☒ Backup restore tested | |

10. CERTIFICATE CONCEPTS

VERIQTA

CONCEPT	DESCRIPTION
API SERVER CERT	Secures communication between clients and API server.
ETCD CERTS	Secure etcd node communication.
KUBELET CERTS	Kubelet authenticates to the API server.
CERT EXPIRY	Expired certs cause API / kubelet failures.
ROTATION	Rotate certs before expiry to avoid outages.



REAL WORLD LESSON:
A BACKUP IS NOT A RECOVERY STRATEGY UNTIL RESTORATION HAS BEEN TESTED.



CONNECT WITH VERIQTA



20. KUBERNETES PRODUCTION COMMAND AND READINESS MASTER SHEET



1. ESSENTIAL COMMANDS



1.1 CLUSTER INSPECTION

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get namespaces
kubectl get componentstatuses
kubectl get csinodes
```

VERIQTA



1.2 POD INSPECTION

```
kubectl get pods -A -o wide
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns>
kubectl logs <pod> -n <ns> -c <container>
kubectl exec -it <pod> -n <ns> -- /bin/sh
kubectl top pod <pod> -n <ns>
```

VERIQTA



1.3 DEPLOYMENT TROUBLESHOOTING

```
kubectl get deploy -A
kubectl describe deploy <deploy> -n <ns>
kubectl rollout status deploy/<deploy> -n <ns>
kubectl rollout history deploy/<deploy> -n <ns>
kubectl rollout undo deploy/<deploy> -n <ns>
```



1.4 NODE TROUBLESHOOTING

```
kubectl describe node <node>
kubectl get nodes -o wide
kubectl top node <node>
kubectl get events --field-selector involvedObject.kind=Node
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>
```

VERIQTA



1.5 NETWORKING COMMANDS

```
kubectl get svc -A -o wide
kubectl get endpoints -A -o wide
kubectl get endpointSlices -A
kubectl get networkpolicies -A
kubectl describe svc <svc> -n <ns>
```

VERIQTA



1.6 DNS DEBUGGING

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl exec -it <pod> -n <ns> -- nslookup <service>
kubectl exec -it <pod> -n <ns> -- dig <service>
kubectl exec -it <pod> -n <ns> -- cat /etc/resolv.conf
```



1.7 STORAGE COMMANDS

```
kubectl get pvc -A
kubectl describe pvc <pvc> -n <ns>
kubectl get pv
kubectl get storageclass
kubectl get csidrivers
```

VERIQTA



1.8 RBAC COMMANDS

```
kubectl get sa -A
kubectl get roles -A
kubectl get rolebindings -A
kubectl auth can-i --list --as <user> -n <ns>
kubectl auth can-i <verb> <resource> -n <ns>
```



1.9 RESOURCE COMMANDS

```
kubectl top pods -A
kubectl describe quota -A
kubectl describe limitrange -A
```

VERIQTA



1.10 LOGS AND EVENTS

```
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
kubectl get events -n <ns> --field-selector type=Warning
```



1.11 ROLLOUT COMMANDS

```
kubectl rollout status deploy/<deploy> -n <ns>
kubectl rollout history deploy/<deploy> -n <ns>
kubectl rollout undo deploy/<deploy> -n <ns>
kubectl rollout restart deploy/<deploy> -n <ns>
```



1.12 SCALING COMMANDS

```
kubectl get hpa -A
kubectl describe hpa <hpa> -n <ns>
kubectl scale deploy/<deploy> -n <ns> --replicas=5
kubectl autoscale deploy/<deploy> -n <ns> --cpu-percent=70
```



1.13 HELM COMMANDS

```
helm list -A
helm status <release> -n <ns>
helm upgrade <release> <chart> -n <ns>
helm rollback <release> <revision> -n <ns>
helm history <release> -n <ns>
```

VERIQTA

2. PRODUCTION READINESS CHECKLIST



2.1 WORKLOAD HEALTH



2.2 REQUESTS AND LIMITS



2.3 PROBE CONFIGURATION



2.4 HIGH AVAILABILITY



2.5 POD PLACEMENT



2.6 PDB VALIDATION



2.7 NETWORK SECURITY



2.8 RBAC VALIDATION



2.9 SECRETS MANAGEMENT



2.10 STORAGE RESILIENCE



2.11 AUTOSCALING



2.12 MONITORING



2.13 LOGGING



2.14 ALERTING



2.15 BACKUP



2.16 DISASTER RECOVERY



2.17 ROLLBACK READINESS



2.18 UPGRADE READINESS



2.19 SECURITY VALIDATION



2.20 CAPACITY VALIDATION

3. FINAL TROUBLESHOOTING FLOW

VERIQTA



TROUBLESHOOTING FLOW PRINCIPLES



Move layer
by layer.



Verify each
component
before moving
forward.



Isolate the
failure
domain.



Check state,
events,
logs, metrics
and dependencies.



Use evidence
to make
correct
decisions.

QUICK COMMAND REMINDER

Get Pods	<code>kubectl get pods -A</code>	Drain Node	<code>kubectl drain <node> --ignore-daemonsets --delete-emptydir-data</code>
Describe Pod	<code>kubectl describe pod <pod> -n <ns></code>	Uncordon Node	<code>kubectl uncordon <node></code>
Logs	<code>kubectl logs <pod> -n <ns></code>	Rollout Status	<code>kubectl rollout status deploy/<deploy> -n <ns></code>
Exec	<code>kubectl exec -it <pod> -n <ns> -- /bin/sh</code>	Scale Deploy	<code>kubectl scale deploy/<deploy> -n <ns> --replicas=5</code>
Get Nodes	<code>kubectl get nodes -o wide</code>	Helm List	<code>helm list -A</code>
Top Pods	<code>kubectl top pods -A</code>	Helm Upgrade	<code>helm upgrade <release> <chart> -n <ns></code>
Get Events	<code>kubectl get events -A --sort-by=.lastTimestamp</code>		



SENIOR ENGINEER RULE:

DIAGNOSE FROM EVIDENCE. CHECK STATE, EVENTS, LOGS, METRICS AND DEPENDENCIES BEFORE CHANGING PRODUCTION RESOURCES.



CONNECT WITH VERIQTA